

华北电力大学

学术硕士学位论文

基于 DPDK 的 APT 攻击检测网关的方法研究

Research on the Method of APT Attack Detection Gateway Based on DPDK

马文杰

2025 年 5 月

国内图书分类号：TP393
国际图书分类号：004.7

学校代码：10054
密 级：公开

学术硕士学位论文

基于 DPDK 的 APT 攻击检测网关的方法研究

硕 士 研 究 生：马文杰

导 师：周蓉副教授

申 请 学 位：工学硕士

学 科 专 业：计算机科学与技术

所 在 学 院：控制与计算机工程学院

答 辩 日 期：2025 年 5 月

授予学位单位：华北电力大学

Classified Index: TP393

U.D.C: 004.7

Thesis for the Academic Master's Degree

**Research on the Method of APT Attack Detection
Gateway Based on DPDK**

Candidate:	Ma Wenjie
Supervisor:	A.P. Zhou Rong
Academic Degree Applied for:	Master of Engineering Science
Speciality:	Computer Science and Technology
School:	School of Control and Computer Engineering
Date of Defence:	May, 2025
Degree-Conferring-Institution:	North China Electric Power University

摘 要

高级持续性威胁（Advanced Persistent Threat, APT）是一种新型网络攻击，具有极强的组织性、隐蔽性、持续性、对抗性和破坏性，目标针对国家重要领域关键基础设施，严重威胁网络安全。针对严峻的网络安全态势，文章分析 APT 攻击检测针对网络设备三个维度难题，提出一种具有软件定义网络（Software Defined Network, SDN）特征的 APT 攻击检测网关架构，利用 SDN 分布式的优势，在网络端点处抵御攻击流量。本文主要工作如下：

（1）针对传统网络架构在面对网络攻击时的局限性，本文通过分析 SDN 新型网络架构的数据平面，基于英特尔（Intel）数据平面开发套件框架（Data plane Development Kit, DPDK）实现网络数据旁路内核协议栈，提高内核稳定性。提出基于 DPDK 的 APT 攻击检测总体架构，通过性能测试验证该架构能够显著提升网络设备性能，具备网络攻击防御基础。

（2）通过分析网络攻击对流表性能的影响，结合 DPDK 特性和多核处理器并行处理网络数据的优势，在总体架构下提出基于 DPDK 的二级核间负载均衡软硬协同架构。硬件提高并行流量处理能力，软件加速流表查找效率。实验使用网络攻击数据集，结合攻击流量特征，通过仿真测试和压力测试验证该架构能够有效提高流表查找效率，具备 APT 攻击纵深防御基础。

（3）通过运用 ATT&CK 模型（Adversarial Tactics, Techniques and Common Knowledge）分析 APT 攻击特征及各个阶段检测难点与目标，在总体架构及软硬协同架构下构建基于 DPDK 的 Snort 入侵检测系统（Intrusion Detection System, IDS）。最后，集成本文研究成果，部署基于 DPDK 的 APT 攻击检测网关，构建 APT 攻击检测集成验证环境，模拟攻击者重放网络攻击，检测网络异常数据流和行为，验证本文架构提高 APT 攻击检测能力同时保障正常网络通信。

关键词：网络安全；DPDK；软件定义网络；高级持续性威胁；负载均衡

Abstract

Advanced Persistent Threat (APT) is a new type of network attack with strong organization, concealment, persistence, confrontation and destructiveness, targeting key infrastructure in national important fields, seriously threatening network security. Aiming at the severe network security situation, the article analyzes the APT attack detection against the three dimensional difficulties of network devices, and proposes an APT attack detection gateway architecture with Software Defined Network (SDN) features, which utilizes the advantages of SDN distribution to defend against the attack traffic at the network endpoints. This thesis is as follows:

(1) Aiming at the limitations of traditional network architecture in the face of network attacks, this paper analyzes the data plane of the new network architecture of SDN and implements a network data bypass kernel stack based on the Intel data plane Development Kit (DPDK) framework to improve kernel stability. The overall architecture of APT attack detection based on DPDK is proposed, and the performance test verifies that the architecture can significantly improve the performance of network equipment and has the basis for network attack defense.

(2) By analyzing the impact of network attacks on flow table performance, combined with the advantages of multi-core processors in parallel processing of network data, a two-level inter-core load balancing hardware and software collaborative architecture based on DPDK is proposed under the overall architecture. Hardware improves the parallel traffic processing capability, and software accelerates the efficiency of flow table lookup. The experiment uses a network attack data set, combined with attack traffic characteristics, and verifies through simulation tests and stress tests that the architecture can effectively improve the efficiency of flow table lookup and has the foundation for in-depth defense against APT attacks.

(3) By using the ATT&CK model (Adversarial Tactics, Techniques and Common Knowledge) to analyze APT attack characteristics and detection difficulties and goals at each stage, a Snort intrusion detection system (IDS) based on DPDK is constructed under the overall architecture and hardware and software collaborative architecture. Finally, the research results of this article are integrated, an APT attack detection gateway is deployed, an APT attack detection integrated verification environment is constructed, attackers are simulated to replay network attacks, abnormal network data flows and behaviors are detected, and the architecture of this article is verified to improve the APT attack detection capability while ensuring normal network.

Keywords: Cybersecurity, DPDK, Software Defined Networking, Advanced Persistent Threat, Load Balance

目 录

第 1 章 绪论.....	1
1.1 课题背景及研究的目的和意义	1
1.2 国内外的研究历史与现状	2
1.2.1 传统网络设备缺陷.....	2
1.2.2 数据平面架构迭代.....	2
1.2.3 高级持续性威胁攻防.....	4
1.3 论文的主要工作.....	4
1.4 论文的结构安排.....	6
第 2 章 基于 DPDK 的 APT 攻击检测总体架构	7
2.1 架构理论基础.....	7
2.1.1 传统网络技术基础.....	7
2.1.2 软件定义网络架构.....	9
2.2 DPDK 技术概述.....	10
2.2.1 DPDK 技术框架及特性.....	10
2.2.2 DPDK 技术在网络数据包处理中的应用优势.....	12
2.3 基于 DPDK 的 APT 攻击检测总体架构设计	13
2.4 网络性能测试实验.....	16
2.4.1 网络性能测试实验环境搭建.....	16
2.4.2 基于 Linux 的 TCP/IP 内核协议栈性能测试实验	20
2.4.3 基于 DPDK 性能测试实验.....	22
2.5 本章小结.....	23
第 3 章 基于 DPDK 的 APT 攻击流量特性研究	24
3.1 流表理论基础及定义.....	24
3.1.1 SDN 流表基础及定义.....	24
3.1.2 基于 TCAM 硬件流表	24
3.1.3 基于哈希算法软件流表.....	25
3.2 基于 DPDK 的二级核间负载均衡软硬协同架构	26
3.2.1 RSS.....	26
3.2.2 布谷鸟哈希	28
3.3 二级核间负载均衡软硬协同架构性能测试实验	31
3.3.1 哈希算法性能仿真测试实验环境搭建	31
3.3.2 哈希算法性能仿真测试实验.....	32
3.3.3 RSS 压力测试实验环境搭建	37
3.3.4 RSS 压力测试实验.....	38

3.4 本章小结.....	40
第4章 基于 DPDK 的 APT 攻击特性研究.....	41
4.1 APT 攻击理论基础及定义.....	41
4.2 APT 攻击检测的目标与挑战.....	42
4.2.1 APT 攻击检测的目标与挑战.....	42
4.2.2 基于 DPDK 的 Snort 入侵检测系统.....	43
4.3 APT 攻击检测仿真实验.....	45
4.3.1 APT 攻击检测仿真测试实验环境搭建.....	45
4.3.2 APT 攻击检测仿真测试实验.....	46
4.4 本章小结.....	49
第5章 结论与展望.....	50
5.1 全文总结.....	50
5.2 研究展望.....	51
参考文献.....	52

第 1 章 绪论

1.1 课题背景及研究的目的和意义

随着网络在人类社会的不断普及和深入，网络环境支撑国家重要领域关键基础设施的高效运行。为了满足当前快速和安全的网络连接，提供的网络服务应该变得更安全、更可靠。近年来，工业系统崩溃、工业敏感信息泄露的风险日益加剧，渗透网络空间造成的破坏对社会各阶层构成了严重威胁。2010 年 9 月 24 日，伊朗核设施曝出震网（Stuxnet）病毒攻击，高度复杂的恶意代码及多个零日（0day）漏洞造成约 1000 台铀浓缩离心机故障，对关键军事设施造成严重破坏为代表的高级持续性威胁事件^[1]。

高级持续性威胁（Advanced Persistent Threat, APT）是指组织良好、技术娴熟的组织利用的一种特定形式手段，针对特定目标实施可持续的新型网络攻击，旨在刺探、收集和窃取核心情报，并监控、渗透和破坏关键基础设施及网络设备^[2]。美国国家标准技术研究所（National Institute of Standards and Technology, NIST）将 APT 攻击定义为“使用大量专业知识和资源的网络攻击，并以各种攻击方法创造机会来实现特定的目标^[3]”。

APT 攻击通常得到寻求获得竞争优势或经济利润的民族国家或有组织犯罪集团的支持。2013 年，斯诺登曝光了美国的“棱镜计划”。曝光显示，自 2007 年起，美国政府对全球实施电子监听，进入微软、谷歌、苹果等多家互联网巨头服务器监控用户隐私资料，其侵犯人群之广、程度之深、时间之长震惊世人。2015 年 12 月 23 日，乌克兰伊万诺弗兰科夫斯克地区遭受 APT 攻击导致大规模停电，该攻击是由于电力系统感染携带 Killdisk 组件的恶意木马 BlackEnergy 所致^[4]。2020 年 12 月 13 日，FireEye 公司发布报告声称 SolarWinds 旗下的软件遭到供应链攻击，黑客通过篡改源代码并添加后门实施网络攻击。2023 年 6 月，卡巴斯基揭露的一起 APT 攻击事件，提到 APT 组织利用了 iOS 系统中多个零日漏洞组合，使用 iMessage 信息服务的 0-Click-0day 在全球范围进行大规模攻击。在奇安信发布的《全球高级持续性威胁（APT）2023 年度报告》中提到 2023 年多个 APT 组织频繁针对国内目标，涉及政府、军工、科研、高校、能源、航天多个领域的重点单位，金融、游戏、媒体、芯片、通信、医疗等行业也遭到 APT 定向攻击^[5]，严重损害了我国国家安全、经济安全、关键信息基础设施安全和广大民众的个人信息安全。

针对当前网络空间面临的现实问题和潜在威胁，2014 年 2 月 27 日，习近平总书记主持召开中央网络安全和信息化领导小组第一次会议并发表重要讲话：

“没有网络安全就没有国家安全，没有信息化就没有现代化^[6]”。

1.2 国内外的研究历史与现状

1.2.1 传统网络设备缺陷

安全一直以来都是全世界人民最为关心的重要问题，近十年来，网络在人们的生活中扮演着重要的角色，各个领域互联网的快速发展，网络安全要求不断提高。保证网络流量安全传输的首选使用虚拟专用网络（Virtual Private Network, VPN），VPN 具有高安全性、高灵活性、低通信成本等特点。VPN 通常使用策略控制、IKE（Internet Key Exchange）协商、隧道加密和密钥管理来保证链路上流量传输的安全。但是，由于系统设计和工程实践等仍会出现诸多不可避免地问题，或着人为造成的缺陷，VPN 技术依然存在诸多漏洞，或者某些 VPN 服务提供商的恶意行为，导致网络资源容易受到网络攻击的困扰^[7]。2021 年发布的第 30 届 USENIX 安全研讨会引入了一种针对 VPN 的新协议级攻击，称为盲入/路径上，分为客户端攻击和服务端攻击，成功地对 VPN 服务器实施了 DNS（Domain Name System）劫持^[8]。服务器端攻击结合加密隧道的流量分析，将伪造的数据包注入 VPN 服务器，巧妙地绕过 VPN 加密隧道造成的攻击屏障，导致 VPN 协议的安全性受到质疑。

针对越来越复杂的网络安全需求，网络行业专家及学者开始迫切寻求技术创新，包括在网络架构层面和应用技术层面等做出体系化革新，满足网络设备对高可靠、高性能、高安全等多方面需求。因此，网络设备厂商也应以更敏锐的洞察速度、更快的响应速度来迭代网络硬件。

1.2.2 数据平面架构迭代

网络领域硬件设备厂商把网络数据处理服务从通用中央处理器（Central Processing Unit, CPU）上卸载下来，将可编程单元作为一种具备独立运作能力的网络处理系统。基于 ASIC（Application Specific Integrated Circuit）的解决方案虽然更快，但缺乏灵活性，并且具有很长的开发周期^[9]。现场可编程门阵列（Field Programmable Gate Array, FPGA）通常用于高速数据包处理中，由于可编程、高通量和低延迟的优势，当数据平面卸载到 FPGA 时，基于 FPGA 的流量监视能够提供准确的流量信息并保持 100Gbps 高吞吐量网络^[10]。但是，FPGA 存在明显短板，功耗高、成本高、设计复杂度高、开发周期长等暴露诸多现实问题，不完全适用于各行业。

传统的网络设备通常采用 CPU 结合内核协议栈，传统的 Linux 协议栈捕获

数据包方式需要进行多次复制，CPU 的处理能力更多用在数据包的拷贝以及转发上，导致数据处理效率较低，在此之上提出一种基于 PF_RING 的电力安全网关的实现方法，通过 PF_RING 框架进行底层数据包捕获和转发，实现了数据快速处理。经过测试表明，性能优于开源框架 Openswan 和 strongSwan^[11]。

早在 2009 年多核处理器进入市场时，学者就验证了多核处理器带来多核的互联网安全协议（Internet Protocol Security, IPSec）并行处理模型性能的大幅提升，其并发处理架构具有良好的可扩展性^[12]。而本文涉及的方案是由英特尔（Intel）公司提出的一个名为数据平面开发套件（Data plane Development Kit, DPDK）^[13]的高速分组数据框架，将通用 CPU 的多个协处理单元结合构成的 NP 系统也可以满足一些边缘网络中的数据包处理需求，具有非常高的灵活性，并且还可以通过固件升级的方式以降低升级成本。

文献基于 DPDK 纯软件方式以构建 VPN 网关，满足可扩展、通用性和高性能的需求，且往返时延（Round-Trip Time, RTT），系统吞吐量，数据包转发率和抖动等性能指标优于其他常见的 VPN^[14]。文献基于移动目标防御（Moving Target Defense, MTD）^[15]理论体系中运用 DPDK 构建的移动 VPN（Moving Virtual Private Network, MVPN）体系结构可以将盲入攻击的盲入攻击成功率降低五个数量级，利用 DPDK 分布式的框架快速实施并部署，MVPN 系统可以有效地防止攻击者在盲入攻击中进行劫持^[16]。文献基于 DPDK 构建网络扫描与其他网络扫描相比，使用较少 CPU 和内存，扫描速度提高 2 倍，识别开放端口精度达到 99.5%^[17]。

SDN 环境下的对网络攻击中，文献对拒绝服务（Denial of Service, DoS）和分布式拒绝服务（Distributed Denial of Service, DDoS）攻击检测溯源算法的准确性和对 DDoS 攻击的抑制程度进行研究^[18]。SDN 数据平面异常行为及相关检测方法展开研究，文献从异常交换机存在场景下的 SDN 数据平面安全性质验证、边缘交换机异常行为及检测方法、核心交换机异常行为及检测方法三个方向进行切入，提高 SDN 数据平面整体安全性^[19]。文献基于 DPDK 框架的流量实时重组及存储技术方案，比相对于原始 Hadoop 分布式文件系统（Hadoop Distributed File System, HDFS）对于小文件的存储效率提升了 13.9%，而对于小文件的读取效率提升了 37.3%^[20]。

网络流表架构设计中，文献基于 DPDK 和赛灵思 U200 硬件加速卡，针对大象流卸载设计实现了高性能软硬协同流表，相较普通哈希（Hash）流表查找性能提升 64%-123%^[21]。布谷鸟哈希（Cockoo Hash）是一种空间高效的近似成员资格查询数据结构，在网络系统中被广泛应用于网络路由、网络测量和网络缓存等，文献设计了指纹可变哈希布谷鸟过滤器从而大幅降低误判代价^[22]。

1.2.3 高级持续性威胁攻防

网络细分邻域电力网络文献基于 ATT&CK (Adversarial Tactics, Techniques and Common Knowledge) 知识库建立了面向新型电力系统 APT 攻击的杀伤链模型并采用 Bert 模型对技术文本进行语义分析, 自动将攻击技术划分到所属阶段, 比现有模型具有更好的效果^[23]。工业物联网 (The Industrial Internet of Things, IIOT) 所面临的 APT 威胁和研究状况, 文献针对 APT 的防御技术使用各种模型来分析 APT 活动的本质, 以提高 IIOT 的安全保护功能^[24]。

归纳总结各阶段的攻击与检测技术, 结合 APT 攻防技术相互博弈、快速发展的趋势, 文献指出了当前攻防双方面面临的挑战和未来研究的发展方向^[25]。面向 APT 攻击多个阶段的网络欺骗防御策略, 文献分别建模分析具体的攻防场景, 并针对性的给出解决方法, 实现 APT 攻击前、中、后三个阶段全过程纵深欺骗防御, 提高防御能力^[26]。位于网络出口 APT 攻击检测系统, 文献根据恶意 DNS 和流量分析有效地检测适当的恶意软件感染, 使用恶意 DNS 分析技术来检测可疑的 APT 恶意软件命令与控制 (Command and Control, C&C) 域, 使用基于签名的基于签名和基于异常的检测技术分析相应可疑 IP 的流量^[27]。

国内外当前网络攻击防御研究主要聚焦在不同攻防场景下的防御策略上, 研究内容相对分散, 总结归纳性研究内容多于实用性研究内容, 缺少对 APT 攻击的实用性防御和流量检测的研究。APT 攻击往往持续时间更长且手段多样, 因此, 也需要多种网络防御方法共同作用才能更好的抵抗 APT 攻击。

1.3 论文的主要工作

目前基于传统 SDN 数据平面的网络设备, 容易受到 APT、DoS/DDoS 攻击、注入攻击等常规网络攻击破坏网络资源, 而内核协议栈存在大量应用将端口暴露在外的问題, 容易受到攻击致瘫网络设备。基于硬件加速数据平面, 往往硬件资源宝贵, 相关资源利用率较低, 且这些平台必须投入大量项目预算, 存在开发周期漫长, 售后保障维护费用高等问题。

针对网络设备受到 APT 攻击时, 文章分析 APT 攻击检测针对网络设备三个维度难题, 因网络架构局限性造成处理性能、系统安全等隐患, 导致 APT 攻击流量处理难题。因流表性能造成软件流表、访问控制、安全策略等查找能力不足, 导致高并发 APT 攻击流量检测难题。因 APT 各个阶段攻击活动的方式不同, APT 多阶段攻击特征及检测难点不同, 导致多阶段 APT 攻击检测难题。

本文针对这些问题, 提出解决方案, 论文的主要工作如下。

1、采用 DPDK 数据平面技术对网络数据加速, 设计基于 DPDK 的攻击检测的总体架构。架构继承 SDN 的核心思想即管理平面和数据平面分离, 功能接

口隔离互联网侧未知安全域与内网侧安全域。数据报文从网卡中接收，再传送到虚拟网络功能（Virtual Network Function，VNF）用户态应用程序处理，网络数据旁路内核协议栈，保障网络设备不易受到常规网络攻击，提高内核稳定性，提高网络数据处理性能，保障网络设备具备处理网络攻击基础。

2、设计网络攻击报文数据集为特征的仿真实验，在既往 Hash 算法的研究基础之上，结合布谷鸟哈希理论，利用多核处理器的高性能分组处理系统，提出基于 DPDK 的二级核间负载均衡软硬协同架构，即 RSS(Receive-Side Scaling) 硬件多核负载均衡和 Cockoo Hash 软件流表组成的混合架构，提高多通道数据包分流能力以及提高并行流量处理能力，实现软件流表项的高效存储和查找。

3、研究高性能网络架构下对 APT 攻击流量的实时检查和分析，运用 ATT&CK 模型从不同维度、不同阶段的攻击进行分析，识别出各层潜在的弱点和漏洞以及面临的 APT 攻击，针对现阶段 APT 攻击检测的目标与挑战，形成 APT 攻击检测解决方案，将 Snort 融入 DPDK 技术架构，结合基于 DPDK 的 APT 检测总体架构，通过 IDS 监控网络流量来检测和分析网络中的异常数据流和行为，实现提高 APT 攻击检测能力的研究目标。

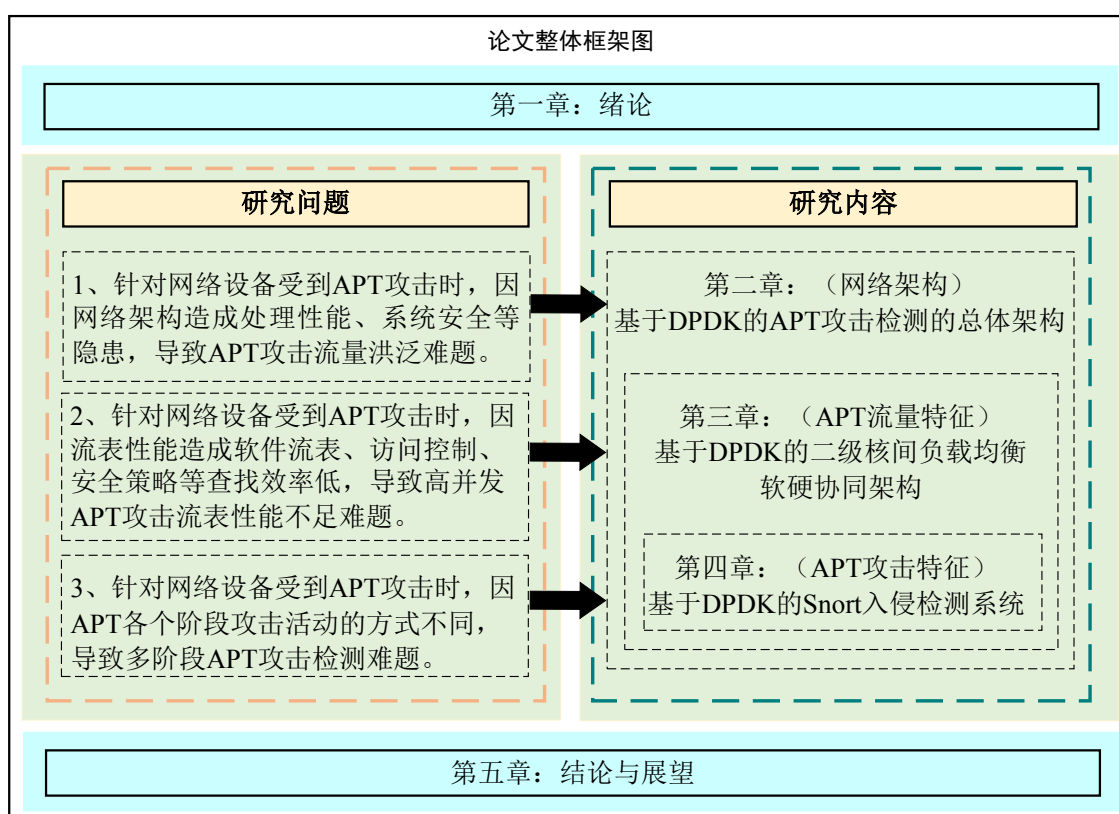


图 1-1 论文整体框架图

论文整体框架如图 1-1 所示，本文研究内容是对现有网络安全技术的研究补充，实验结果能够为网络设备制造商、网络运营商以及科研机构等提供数据参考，是对未来网络空间安全领域防护能力提升的重要探索。

1.4 论文的结构安排

本文剩余部分的结构安排如下：

在第二章中，本章分析传统网络架构在面对网络攻击时的局限性，分析网络攻击对网络性能影响，简要介绍传统网络架构到现代 SDN 新型网络架构，根据 APT 网络攻击环境下网络设备性能需求，重点研究数据平面，采用 DPDK 实现网络数据旁路内核协议栈，提高内核稳定性，提出基于 DPDK 的 APT 攻击检测总体架构，通过 RFC2544 网络性能测试实验，验证该架构能够显著提升网络设备性能，具备网络攻击防御基础以及扩展性。

在第三章中，本章分析网络攻击流量对流表项性能影响，简要介绍现代网络设备流表的理论基础以及定义，分析目前常用的基于硬件流表项存储方案和基于软件流表项存储方案优劣势。结合 DPDK 技术特性和多核处理器并行处理网络数据的优势，结合第二章 APT 攻击检测的总体架构下提出基于 DPDK 的二级核间负载均衡软硬协同架构。硬件提高并行流量处理能力，软件加速流表查找效率。使用网络攻击报文数据集，结合 APT 攻击流量特征对八种 Hash 算法仿真压力测试，以及 RSS 压力测试，验证该架构能够有效提高流表查找效率，具备攻击流量检测的优势，具备 APT 攻击纵深防御基础。

在第四章中，本章首先简要介绍本文所涉及的 ATT&CK 模型有关 APT 攻击的理论基础及定义，通过运用 ATT&CK 模型分析 APT 攻击特征及各个阶段检测难点与目标，在总体架构及软硬协同架构下构建基于 DPDK 的 Snort 入侵检测系统。最后集成第二章到第四章研究成果，部署基于 DPDK 的 APT 攻击检测网关，构建 APT 攻击检测集成验证环境，使用 APT 攻击报文数据集模拟攻击者重放网络攻击，检测网络异常数据流和行为，验证本文架构提高 APT 攻击检测能力同时保障正常网络通信。

在第五章中，对全文的主要工作以及结果进行简要的总结，并叙述了对后续研究工作的建议与设想。

第 2 章 基于 DPDK 的 APT 攻击检测总体架构

2.1 架构理论基础

2.1.1 传统网络技术基础

近十几年来，互联网在人们的生活中发挥着重大作用，各个领域对互联网的需求迅速增加。为了能够快速、安全地连接到互联网，提供服务的网络必须更加快速、可靠。自 20 世纪 70 年代以来，为了实现上述目标，人们提出了许多网络数据通路。它起始于传输控制协议（Transmission Control Protocol, TCP）和用户数据报协议（User Datagram Protocol, UDP），后来又出现了几种更现代的路径，包括快速 UDP 网络连接（Quick UDP Internet Connections, QUIC）、远程直接内存访问（Remote Direct Memory Access, RDMA）^[28]。

自上个世纪以来，世界上已知的最大的计算机网络就是互联网。互联网在人类生活的各个方面都发挥着关键作用。各个领域的网络应用呈爆炸式增长，以支持不断增长的利用网络的需求。OSI（Open System Interconnect）模型是 ISO 组织在 1985 年研究的网络互连模型。该体系结构标准定义了网络互连的七层框架如图 2-1 所示，网络模型每一层都执行特定的功能。为了使网络体系结构更加清晰、可追溯，并提供一致、可靠的服务，将网络模型划分为网际互连协议（Internet Protocol, IP）五层模型或 TCP/IP 四层模型。

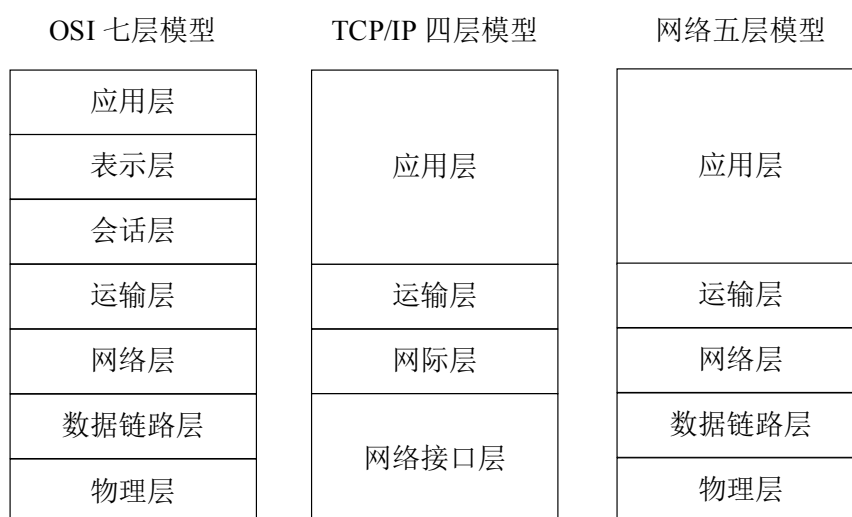


图 2-1 网络的体系结构

网络系统试图确保数据包在传输之前是无差错的，并且所有的数据包都到达并正确地重新组合到目的节点。传输层允许进程进行通信，并控制进程到进程的交付，应用数据以段的形式出现，为传输做好准备，所有的片段将在到达

目的地时重新组合。在传输层起作用的协议根据连接分为面向连接的和无连接的。面向连接的协议首先建立一个虚拟连接，然后将连接从源节点转移到目的节点，一旦所有的数据都被转移，连接就会终止。这类协议通常采用一定的机制来控制流量和传输错误^[29]。另一方面，无连接协议在发送数据之前不建立连接，每一个单独的段都被当作一个独立的数据包。使用这些协议传输的数据被盲目地传送到目的进程，而没有任何流量或差错控制机制。TCP 和 UDP 是面向连接和无连接协议中最著名的例子。

在硬件方面，基于专用中间件的网络设备也是网络系统的重要组成部分，对网络系统的性能和可靠性有着重要的影响。这些设备的主要任务是转发和过滤数据包，但它们支持许多其他网络功能（Network Function, NF），包括服务质量（Quality of Service, QoS）、深度包检测（Deep Packet Inspection, DPI）和数据加密服务等。然而，基于硬件的网络设备存在一些弊端。首先，基于硬件的网络设备价格相对昂贵。对于数量巨大的这些设备，购买和供电成本是无法承受的。其次，基于硬件的网络设备的管理有一定的难度，因为管理员必须为不同厂商提供的设备学习不同的网络操作系统。第三，中间盒的种类日益增多，使得管理员掌握其新的操作系统变得更加困难，基于硬件的网络设备空间消耗大，灵活性差^[30]。

2012 年，许多大型运营商提出网络功能虚拟化（Network Functions Virtualization, NFV）的概念，旨在将 NF 从专用硬件设备中解耦出来^[31]。这种方法可以将 NF 转移到现成的服务器上运行的软件。然而，从硬件到软件的转变并不是一蹴而就，最大的障碍是软件性能与硬件性能之间的差距。基于 Unix 的软件路由器具有很高的灵活性和较低的成本，但是数据包处理速度较慢，巨大的延迟对于网络来说是不可接受的。为此，谷歌提出 QUIC 协议，这是一种加密和多路复用的传输协议。另一种协议采用 RDMA 技术，允许在不涉及其他网络设备操作系统的情况下，从其他网络设备的已知内存区域写入、读取数据。

但是，传统的可编程网络设备通常采用通用 CPU 结合 TCP/IP 内核协议栈，基于通用 CPU 可移植可编程的优势，实现对网络数据报文的处理，凭借 Linux 内核强大的扩展能力，使网络设备能够适应不断涌现的不同网络体系场景。但是，面对当前呈现出指数级增长的网络带宽需求态势，受摩尔定律面临的瓶颈影响，通用 CPU 的处理速度提升幅度相对较小。2020 年之后的网络设备，已从 10Gbps、40Gbps 开始向 100Gbps-800Gbps 的超高速网络迈进，同时在流量密集型骨干网络环境中，网络带宽需求甚至突破 1.6Tbps。因此，以 Linux 内核协议栈搭配通用 CPU 为核心设计实现的网络设备难以满足日益增长的高速网络需求。这些趋势充分凸显了传统网络架构在高性能网络设备设计中的局限性。

综上所述原因，设计下一代网络设备架构时，不仅需要考虑具备更高规格

的网络防御能力，还应该需要采用具备更高数据处理性能的架构。将传统的通用 CPU 的内核协议栈进行旁路处理，这种技术路线已经成为下一代网络设备设计提升整体综合网络性能的解决方案之一。

2.1.2 软件定义网络架构

随着快速发展的网络架构体系，新型网络架构 SDN 作为其中代表，已经在多个行业的核心网络系统中得到了应用。SDN 的核心思想是将控制平面与数据平面分离，此方法降低了网络的整体复杂性并增强可扩展性，并且提高了网络的开放性和可编程性。基于软件的控制器有效地管理网络流量，直接与物理硬件进行交互。这种方法获得全球吸引力，并在研发上进行了大量投资。

SDN 由三个平面组成：应用平面（Application Plane, AP）、控制平面（Control Plane, CP）和数据平面（Data Plane, DP），如图 2-2 所示。

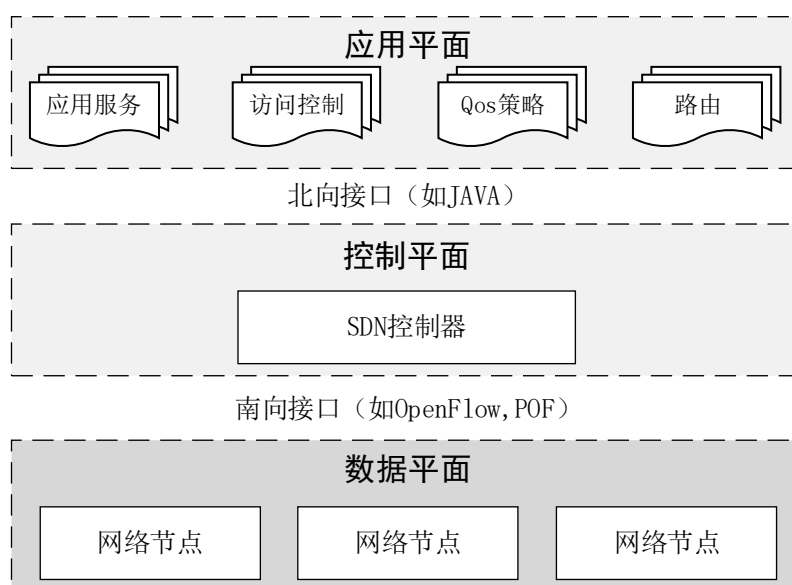


图 2-2 软件定义网络（SDN）结构

应用平面包括网络虚拟化、入侵检测系统（Intrusion Detection System, IDS）和利用虚拟化的移动性管理等网络应用，通过北向应用接口与控制平面交互的移动性管理等，为网络管理员和应用开发人员在控制平面使用 SDN 服务实现重要网络管理提供了一种稳定、一致的方式。控制平面高效地使用 SDN 服务来实现重要的网络管理。作为 SDN 架构中核心的部分，它提供了对网络的细粒度控制，并提供了包括路由计算、监控、网元端多种网络服务。控制平面将这些服务从应用平面转化为一套清晰的指令，并将其下发到相应的数据平面设备中，可以对整个网络进行配置，可以扩展到分布式设备。

数据平面通常被用作简单的转发单元，无需软件扩展到分布式或多个控制器。对于数据平面而言，它们被用作做出更即时的决策，更快速的转发。因此，

网络智能化被通常转移到能够做出决策的应用平面和控制平面上。但是，对于任何一个网络决策都请求到控制平面，以及控制指令转换到应用服务时，网络的安全性和通信的效率将大大降低。本文重点研究了数据平面，保证数据平面基础转发性能同时，提高网络智能化，在网络端点处抵御网络攻击。

在 SDN 环境下，攻击检测技术的改进研究需要考虑以下几个方面：一是利用 SDN 的全局视图优势，结合先进的数据分析技术，提高检测的准确性和实时性；二是针对 SDN 的特点，设计新的攻击检测算法，减少误报和漏报；三是考虑网络的动态变化，开发动态调整的防御策略，以快速响应网络攻击事件。

当前的研究不仅集中在提升检测技术的性能，还包括了攻击溯源和缓解机制的发展。通过结合 SDN 的特点，研究人员提出了多种新的检测与防御方案，如基于信息熵和网络自相似性的检测方案，以及基于区块链的协作式防御方法，这些方案旨在通过人工智能实现对网络攻击的快速、准确识别和高效防御。

总体而言，本章针对传统架构面临攻击流量洪泛时消耗大量物理资源、内核上下文切换等耗时环节，导致网络数据报文处理性能不足，应用 SDN 架构网络攻击威胁。SDN 环境下的网络攻击检测技术正处于快速发展阶段，未来随着技术的进一步发展，新的技术和方法不断涌现，以期解决传统方法的不足。

2.2 DPDK 技术概述

2.2.1 DPDK 技术框架及特性

DPDK 是由 Linux 基金会托管的一个开源项目，用于加速在各种 CPU 架构上处理网络数据包的框架组成。网络性能、吞吐量和延迟对于各种应用程序都至关重要，包括无线和有线基础设施、路由器、负载平衡器、防火墙、视频流和 VoIP (Voice over Internet Protocol)。DPDK 提高了 Intel x86、ARM 和 PowerPC 等各种 CPU 架构上的数据包处理速度，DPDK 框架是快速开发高速数据包网络应用程序的关键。同样，DPDK 框架允许开发人员编写自己的网络协议栈并在用户空间中实现一组高性能数据报转发。

DPDK 框架在内核态 (Linux Kernel) 提供了两个关键的模块：UIO (Userspace I/O) 和 KNI (Kernel NIC Interface)。KNI 模块提供用户态到 Linux 内核态的协议栈的通道，适配 Linux 传统的网络工具例如 ethtool 等实现传统网络功能管理。UIO 模块将网卡硬件寄存器通过内存映射到用户态，形成 DPDK 专用网卡驱动 IGB_UIO，例如 Intel 的 DPDK 网卡驱动 igb_uio.ko。

通过运行 DPDK 可以显著加速其网络应用程序的性能，这要归功于其高效的运行到完成模型和优化的库，这些库可确保预先分配所有必要的资源。在 DPDK 中，库和驱动程序构成了较低层，具有在英特尔架构上的数据平面应用

程序中高速处理数据包所需的功能。DPDK 运行在 Linux 操作系统上，完全替代网络堆栈。它正在实现运行到完成模型，其中必须在数据平面应用程序在处理器内核上运行之前分配所有资源。DPDK 功能官方框架模型如图 2-3 所示。

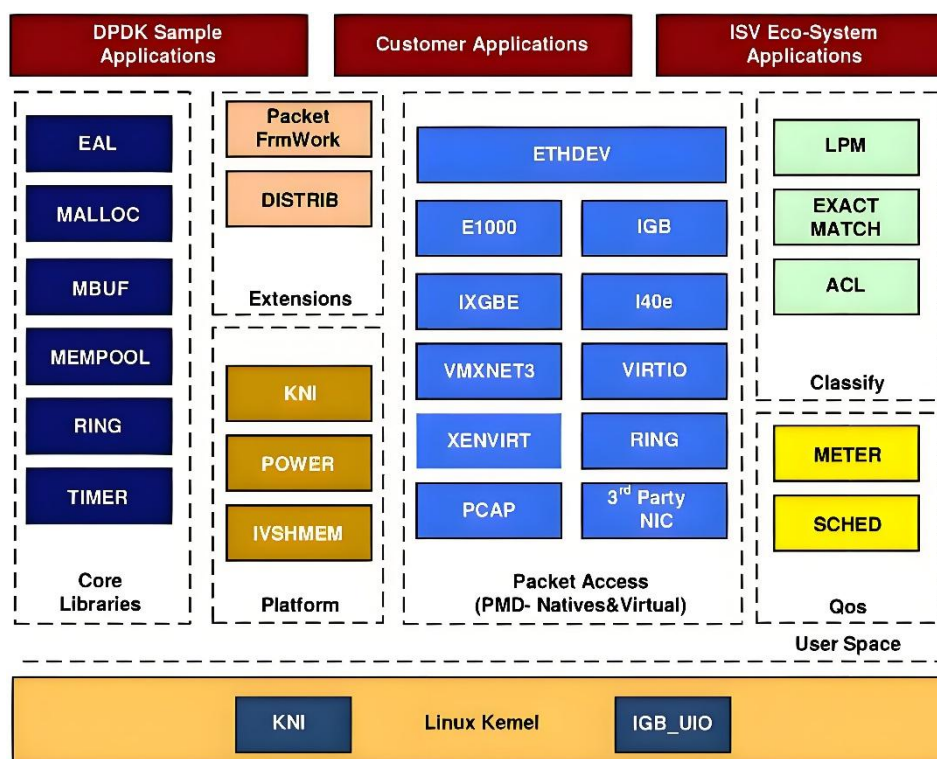


图 2-3 DPDK 官方框架模型

(1) **环境抽象层 (Environment Abstraction Layer, EAL)** EAL 必须能够访问所有低级资源（例如硬件和内存）才能创建所需的接口。在此阶段，所有应用程序都可以启动和加载，所有内存空间处理都可以完成，并且所有高速数据包处理功能都已准备就绪。DPDK 不提供处理 Linux 网络堆栈完全支持的防火墙或协议（例如 TCP 或 UDP）的功能。为了使数据包处理速度更快，DPDK 使用一组针对高性能进行优化的库。这些库在用户空间中执行，并执行与 Linux 网络堆栈类似的所有基本任务，例如缓冲数据包描述符、为数据包分配内存以及通过 NIC 将数据包传递给应用程序，反之亦然。

(2) **队列管理** 提供的功能通过提供固定大小的环形结构和遵循 FIFO (First Input First Output) 原则的双链表无锁实现入队和出队标准功能来管理任何类型的队列。

(3) **内存管理** 物理内存的映射由 EAL 处理，其中每个对象和结构都被授予物理内存的特定部分。为了实现最大性能，在对象之间添加合适的填充以确保它们均匀加载。

(4) **缓冲区管理** DPDK 传输网络数据包提供了所需的缓冲区，这些缓冲区是在任何应用程序运行时创建的。在运行时期间，用户必须能够选择和使用

任何可用的缓冲区，并且之后必须释放它。缓冲区的大小通常很小，但如果传输较大的数据包，则会链接多个缓冲区。DPDK 管理和分配可用缓冲区，DPDK 预分配固定大小的缓冲区，减少了操作系统在分配和释放缓冲区上的耗时。

(5) 其他 例如最长前缀匹配 (Longest Prefix Match Algorithm, LPM)，LPM 有助于实现基于 IPv4 地址转发数据包的算法表。此外 DPDK 支持哈希，使用布谷鸟哈希实现，用于在大量条目中查找特定唯一键。

2.2.2 DPDK 技术在网络数据包处理中的应用优势

由于软件交换、路由技术在 NFV 系统中的核心作用，在网络高速转发场景下，限制系统性能瓶颈的因素之一是数据报文转发能力。数据报文从网卡接收后，内核态通过上下文切换传输数据至用户态应用程序处理过程中，消耗大量资源用于物理 I/O 与地址映射转换、CPU 软硬中断处理、内核协议栈处理、内核上下文切换以及内存拷贝等 CPU 密集型操作和 I/O 处理环节。

行业常用优化方案为旁路内核协议栈、多核处理器负载均衡、消除海量中断、Intel VT (Intel Virtualization Technology) 虚拟化技术等手段对普通开发人员来说具有一定的难度。全面提升数据平面报文处理性能。为了更好地满足商业环境支持及行业需求，提供友好的用户开发环境，DPDK 技术是典型解决方案之一。在 LTE 测试环境模拟 5G 网络的 LTE 测试结果中，DPDK 改进端到端延迟，超低时延表明 DPDK 改良的中间件在未来的 5G 场景中发挥重要作用^[32]。DPDK 技术在网络数据包处理中特性有如下应用优势。

(1) CPU 亲和性 (CPU Affinity) 操作系统通过分时调度机制，管理任务调度确保各进程或线程能够高效运行，切换前将处理器的状态寄存器保存在堆栈中，切换后恢复原线程的状态信息，状态切换时产生额外的性能消耗，同时切换后调度到其它核的线程缓存数据将完全清除，导致数据重复缓存 CPU 利用率降低。将指定线程绑定到指定 CPU 核心上运行，避免操作系统切换线程导致迁移到其它核心运行过程中带来的额外性能开销，DPDK 多线程在多核处理器的指定核心执行，扩展基于内核的 pthread 线程库，将 DPDK 指定线程和 CPU 指定核心绑定，DPDK 指定线程可以独占 CPU 资源处理网络数据报文，避免因多核调度切换而带来的性能瓶颈。

(2) 轮询技术 传统网卡在接收或发送网络报文时，网卡硬件会通过产生硬件中断，网络数据报文经过 Linux 内核协议栈，通过中断上下文通知应用程序处理网络数据报文。Linux 操作系统架构处理硬件中断，处理器的状态寄存器保存到程序堆栈中，处理中断至少需要约几百个时钟周期，执行中断服务程序完成后恢复原线程状态信息。海量中断导致高性能网络处理数据报文时产生

巨大的开销。DPDK 采用轮询技术减轻中断延迟，通过将网卡硬件与用户态应用程序建立内存映射关系，避免了采用中断方式处理网络报文造成的响应延迟，通过 PMD（Poll Mode Driver）轮询方式进行网卡报文收发，极大程度提高网卡收发性能。PMD 模块不仅支持物理网络接口，还支持虚拟化网络接口，不仅支持 Cisco、Broadcom、Mellanox、Intel 等国外网卡芯片，目前最新版本支持 Huawei、Wangxun 等国产网卡芯片，覆盖整个行业生态系统。轮询模式下网卡收到报文后，通过处理器 fetch 指令直接放置到处理器缓存，没有 DMA 技术的情况下，设置报文到达的标志位并保存到内存中。PMD 轮询过程中完全没有中断产生，上层协议软件只需要检测报文到达的标志位，从而判断缓存中的报文是否需要处理，显而易见提升网络数据处理效率。

（3）大页技术 内存管理是现代处理器核心机制一，处理器将内存划分为物理内存和虚拟内存进行管理。Linux 操作系统按帧（Frames）管理物理内存，按页（Page）管理虚拟内存。内存管理单元（Memory Management Unit, MMU）负责将虚拟内存地址转换为物理内存地址，页表存储虚拟地址到物理地址的映射信息。页表查找消耗大量 CPU 资源，Intel 处理器芯片引入 TLB（Translation Lookaside Buffer）缓存减少页表查找时间，TLB 临时存储最近使用过的地址转换记录。当虚拟地址转换为物理地址时，处理器优先在 TLB 中查找是否存在有效历史映射信息，未找到有效映射关系即产生 TLB miss，处理器再查找页表，因此高性能网络架构需要避免 TLB miss。Linux 操作系统默认页表大小是 4K，使用了大页技术后 TLB 表项能够覆盖更大范围内存空间，例如配置 1G 以上的巨页，从而显著降低 TLB miss 的频率。早期的 Linux 内核版本没有为硬件提供大页技术，Linux 内核版本 2.6 以后的版本引入 Linux 大页文件系统特性，允许应用程序直接预先分配大块内存。这种优化为后续的 NFV 等应用提供了更坚实的基础。DPDK 环境中大页技术被用于优化网络数据包处理，所有资源从内存池分配，通过 mbuf 缓存的高效管理和分配，从而提升了网络数据链路的整体性能。

2.3 基于 DPDK 的 APT 攻击检测总体架构设计

该方案通过探索和实现基于 DPDK 技术的 APT 攻击检测方法，旨在保证网络系统基础转发能力下，在网关端点处提高攻击检测的准确性和网络容量，增强对 APT 攻击的快速响应能力，而基于 DPDK 的网络设备形态统一，软件功能部署灵活，具有较高研究、应用价值。

面对国内外严峻的国际形势，各方迫切大力发展自主可控、安全的网络基础设施和信息系统，彻底实现网络安全必须要先实现网络硬件和网络设备架构

的自主可控，以抵御具备多样性、隐蔽性特征 APT 攻击威胁。基于 DPDK 的 APT 攻击检测网关，CPU 硬件核心部件使用国产申威 3231，SMP 架构，Core3B 核心，主频 2.4Ghz，采用自主可控的 SW64 指令集，内部集成 32 个处理核心。32 个处理核心能够最大程度发挥出 DPDK 多核并行的处理能力。“国之重器”神威太湖之光超级计算机使用的 CPU 同样也是中国自主研发的申威 26010 处理器。操作系统使用基于 Linux 的国产操作系统统信 UOS（Unity Operating System），UOS 由深度操作系统为基础打造的国产替代操作系统。

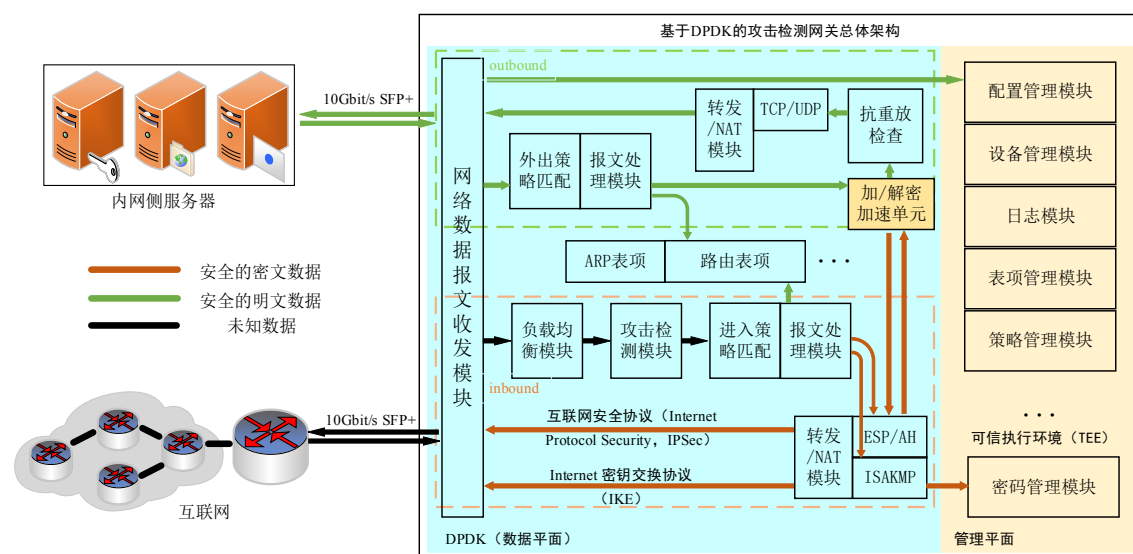


图 2-4 基于 DPDK 的攻击检测网关总体架构

基于 DPDK 的攻击检测网关总体架构如图 2-4 所示，架构继承 SDN 的核心思想即管理平面和数据平面分离，功能接口隔离外网侧未知安全域与内网侧安全域。架构内部各重点模块的功能描述如下。

（1）**网络数据报文收发模块** 该模块是架构的核心模块之一，本文第二章重要研究对象。该模块基于 DPDK 高速处理数据包加速架构，负责接收和发送来自内部域和外部域的数据包。数据报文从互联网侧未知安全域发出，通过 10Gbit/s 光纤接口进入网络数据报文收发模块接收端，将光信号转换成电信号，通过 igb_uio 驱动模块，由网卡直接内存访问（Direct Memory Access, DMA）通道将数据传输到 DPDK 进程。同理，数据报文从内网侧安全域发出传输到 DPDK 进程。数据报文从 DPDK 发送队列发送时，通过 igb_uio 驱动模块把数据报文传输到网卡，电信号转换成光信号，通过 10Gbit/s 光纤接口传输报文。

（2）**负载均衡模块** 该模块是架构的核心模块之一，即 RSS，本文第三章重要研究对象。数据报文经网络数据报文收发模块传输到 DPDK 进程后，进入负载均衡模块，根据报文五元组特征（原 IP 地址、目的 IP 地址、协议号、原端口号、目的端口号），基于网卡硬件负载均衡规则，每个核均衡独立处理到达该队列的报文，分流不同数据流到多个接收队列的 CPU 核的数据处理线程中进

行处理，提高了并行数据处理效率。

(3) **攻击检测模块** 该模块是架构的核心模块之一，本文第四章重要研究对象。不同 CPU 核的数据处理线程的攻击检测模块收到数据报文后，将通过分析网络流量的有效负载来成功检测数据流量的方法，对 APT 攻击前期流量进行分析，识别攻击并阻断。

(4) **进入/外出策略匹配模块** 该模块是架构的核心模块之一，即流表，本文第三章重要研究对象。通过 APT 攻击前期攻击检测模块，根据绑定到 DPDK 的接口接收到的传输流量，根据数据包进入或外出的传输方向和五元组特征(原 IP 地址、目的 IP 地址、协议号、原端口号、目的端口号)，通过 Hash 流表项查找的方式匹配对应索引安全联盟 (Security Association, SA) 执行对应隧道的策略和策略参数，将数据报文传递给相应的报文处理模块。

(5) **报文处理模块** 该模块是对数据报文处理的模块，涉及到对网络报文的封装、解封装及二层信息处理。以 IPSec 协议为例，IPSec 中通过加密技术提供机密性和完整性，该 IPSec 报文处理模块将 ESP (Encapsulating Security Payloads) 数据包分发到加解密模块进行加解密和验证，或者密钥协商 AH (Authentication Header) 报文，该模块将数据报文分发密码管理模块进行 IKE 协商流程以及密钥分发同步。

(6) **密码管理模块** 该模块负责进行 IKE 协商流程以及密钥分发同步。该模块位于管理平面，将密钥等重要信息从到活跃的数据平面处理节点转移到可信执行环境 (Trusted Execution Environment, TEE) 的安全环境中。密钥可基于安全套接层 (Secure Socket Layer, SSL) 或者基于中国国家密码管理局于 2007 年颁布的新一代自主研发密码技术 SM 系列国密算法或基于其他加密协议进行传输，以避免密钥泄露，保证密钥信息的绝对安全。

(7) **加/解密加速单元** 该单元由专用芯片独立组成，存在多个加密-解密节点组成，保证加解密信息的绝对安全，以总线形式连接，仅对传入的数据包进行加速加密、解密及多节点验证的功能。

(8) **转发/NAT 模块** 该模块维护网络地址转换状态流表项 (Network Address Translation, NAT)、二层转发流表项，支持 IPv4/IPv6 报文转发，支持静态配置和动态学习媒体存取控制 (Media Access Control, MAC) 地址，支持基于虚拟局域网 (Virtual Local Area Network, VLAN)。支持 Access、Trunk、Hybrid 模式，支持静态和链路聚合控制协议 (Link Aggregation Control Protocol, LACP) 动态端口汇聚。支持链路层发现协议 (Link Layer Discovery Protocol, LLDP)，支持地址解析协议 (Address Resolution Protocol, ARP)，生成树协议支持 802.1D 生成树协议 (Spanning Tree Protocol, STP)、802.1W 快速生成树协议 (Rapid Spanning Tree Protocol, RSTP)。

(9) **配置管理模块** 该模块主要以配置数据库为中心进行管理，通过内部域网络访问配置管理网页，实现管理平面配置管理，涉及到接口配置、网络信息配置、用户信息配置及其他配置管理操作。

(10) **设备管理模块** 该模块主要管理设备状态，例如网络流量统计、网络实时带宽、流表项查询、CPU 使用率以及设备电压状态等信息。

(11) **日志模块** 该模块主要记录各种级别设备日志，支持操作日志和系统日志、分级告警，例如严重错误（Fatal）事件，Fatal 事件可能导致程序中断。错误（Error）事件，Error 事件不会导致程序中断。潜在有害（Warning）事件。一般（Info）信息，Info 信息描述程序运行过程。调试（Debug）信息，用于诊断问题。跟踪（Trace）信息是最详细的调试信息，Trace 程序执行过程。

(12) **策略管理模块** 该模块主要管理策略。例如访问控制列表（Access Control List, ACL）规则策略，安全策略库（Security Policy Database, SPD）策略，APT 攻击检测策略等，并支持策略同步、实时查询。

(13) **表项管理模块** 该模块主要管理流表项。例如路由表或称路由择域信息库（Routing Information Base, RIB）支持静态路由及开放式最短路径优先（Open Shortest Path First, OSPF）、边界网关协议（Border Gateway Protocol, BGP）的路由协议，安全联盟数据库（Security Association Database, SAD）等，并支持 NAT 状态流表项、二层转发流表项同步、实时查询。

架构还具有基础网关设备功能，本文不深入研究，列举如下：支持动态主机配置协议（Dynamic Host Configuration Protocol, DHCP）。管理与维护支持 SSH（Secure Shell），支持 OAM（Operation Administration and Maintenance）功能，支持 Ping（Packet Internet Groper）、Traceroute，支持 Openflow、Netconf 等 SDN 特性，SDN 集中控制、本地控制，能够满足各类 SDN 应用场景的需求。

2.4 网络性能测试实验

2.4.1 网络性能测试实验环境搭建

网络性能测试实验由于对比不同数据平面网络性能差别，实验环境采用相同硬件运行环境，相同操作系统，基于不同转发架构，使用专业网络测试工具测出实验数据，硬件运行环境见表 2-1。

表 2-1 硬件运行环境与测试单元的关系

CPU	DDR4	操作系统	测试单元
国产申威 3231CPU	16GB	统信 UOS	网络性能测试
DPDK 使用 GNU 编译器套件（GNU Compiler Collection, GCC）、ninja、			

meson 工具进行编译，配置操作系统巨页，在 GRUB_CMDLINE_LINUX 参数后追加 default_hugepagesz=1G hugepagesz=1G hugepages=4，加载 igb_uio.ko 官方网卡驱动，运行在统信 UOS 操作系统上，软件运行环境见表 2-2。

表 2-2 软件运行环境与功能的关系

软件运行环境/依赖	功能	软件/内核版本
GCC	编译工具	8.3.0
Python	脚本工具	3.10.12
DPDK	数据平面	19.11
L3fwd 转发实例	三层网络转发软件	DPDK-19.11
统信 UOS V18	操作系统	Linux 4.19.0-sw64-server #225 SMP sw64 GNU/Linux

接口传输速率为 10Gbit/s，使用华为 10Gbit/s SFP+光模块，将发送端把电信号转换成光信号，数据通过 850nm 多模光纤传输到接收端，接收端再把光信号转换成电信号，使用华为 S5700 交换机连接整个实验网络。主要网络性能测试相关设备见表 2-3。

表 2-3 主要网络性能测试相关设备

测试仪器	功能	软件版本/硬件型号
Spirent TestCenter	网络测试仪	Spirent TestCenter C1
信而泰 BigTao220	备用网络测试仪	板卡类型：V8008D 8x10GE Fiber Renix 测试软件：3.0.8.826351
华为 10Gbit/s SFP+光模块	光模块	模块类型：SFP+，光纤多模 LC 传输速率：10Gbit/s，距离 0.3km
多模光纤	线缆	接口标准：10GBASE-SR 光纤传输模式：多模 LC： 光纤外径：2mm，长度：5m
终端 PC (Personal Computer)	操作、控制仪器	Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz
华为 S5700 交换机	交换机	FutureMatrix S5700 V200R023C00

在性能测试实验中，实验环境的网络拓扑如图 2-5 所示。将网络测试仪和待测试的攻击检测网关通过光纤连接，网络测试仪外出方向端口经交换机连接到攻击检测网关进入方向端口，攻击检测网关外出方向端口直连网络测试仪进入方向端口。将网络测试仪配置网口、攻击检测网关配置网口、终端 PC 的配置网口通过管理交换机连接，使用终端 PC 进行配置操作。

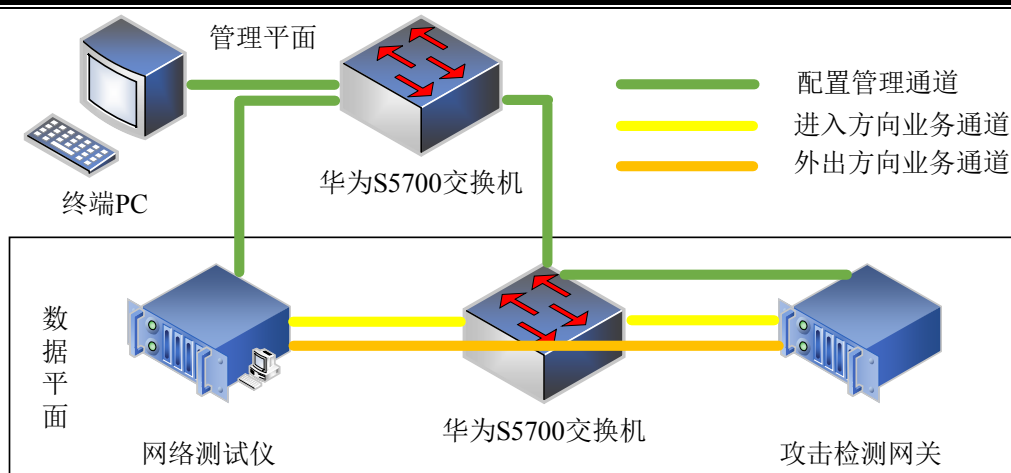


图 2-5 性能测试网络拓扑

网络测试仪会在发送的数据包中加入数据报文的字节校验标志，并在接收数据的时候网络测试仪会通过校验标志验证数据报文的正确性。该项性能测试主要对以下几个网络性能指标进行测试^[33]：

- （1）负载（OfferedLoad） 网卡当前流量占端口速率百分比。
- （2）时延（Latency） 数据从发送到接收端所需的时间。算法 2-1 所示。

算法 2-1.时延测试算法

输入：发送速率 R ，测试帧长 L ，测试轮数 N ，测试时间 T ；

输出：时延 LA 。

- 1.初始化局部变量发送时间 $T1$ ，接收时间 $T2$ ，测试计数 n ；
- 2.WHILE 测试轮数 $n < N$ ；
3. 以 R 发送速率发送 T 时间，记录发送时间 $T1$ ，接收时间 $T2$ ；
4. 本轮时延 $LA = T2 - T1$ ；开始下一轮；
- 5.RETURN 测试 N 轮平均时延 LA 。

（3）丢包率（Loss Tolerance） 传输过程中丢失的数据包占总发送数据包的比例，是衡量网络性能和可靠性的一个重要指标。算法如 2-2 所示。

算法 2-2.丢包率测试算法

输入：发送速率 R ，帧长 L ，测试轮数 N ，测试时间 T ；

输出：吞吐量 $Loss$ 。

- 1.初始化局部变量发送测试包数量 $P1$ ，接收测试包数量 $P2$ ，测试计数 n ；
- 2.WHILE 测试轮数 $n < N$ ；
3. 以 R 发送速率发送 T 时间，共计发送测试包数量 $P1$ ，接收测试包数量 $P2$ ；
4. 本轮丢包率 $Loss = (P1 - P2) / P1$ ；开始下一轮；
- 5.RETURN 测试 N 轮平均吞吐量 $Loss$ 。

(4) 吞吐量 (Throughput) 数据包每秒的收、发个数。算法如 2-3 所示。

算法 2-3.吞吐量测试算法

输入：初始发送速率 R ，帧长 L ，测试轮数 N ，测试时间 T ，吞吐量测试精度 A ；

输出：吞吐量 TP 。

- 1.初始化局部最大发送速率 R_{max} ，最小发送速率 R_{min} ，测试计数 n ；
- 2.初始化局部变量发送测试包数量 $P1$ ，接收测试包数量 $P2$ ；
- 3.WHILE 测试轮数 $n < N$ ；
4. 以 R 发送速率发送 T 时间，共计发送测试包数量 $P1$ ，接收测试包数量 $P2$ ；
5. IF $P1 = P2$ THEN 表示没有丢包，则提高发送速率 R ， $R_{min} = R$ 开始下一轮；
6. IF $P1 > P2$ THEN 表示出现丢包，则降低发送速率 R ， $R_{max} = R$ ；
7. IF $R_{max} - R_{min} \leq A$ THEN 本轮吞吐量 $TP = (R_{max} + R_{min}) / 2$ ；
8. IF $R_{max} - R_{min} > A$ THEN 开始下一轮；
- 9.RETURN 测试 N 轮平均吞吐量 TP 。

网络测试仪使用思博伦测试仪 (Spirent TestCenter)，辅助使用国产信而泰 BigTao220 网络测试仪对测试结果二次验证。网络测试仪能够在 10M 至 100G 之间的多种速率下，具体取决于网络接口速率，可以构造多种网络协议（如 IP，DHCP，TCP 与 UDP 等）进行测试。它还支持对协议报头的部分内容（如源目的 IP、源目的 MAC 等）的用户自定义以及收发前后的数据报文统计功能。

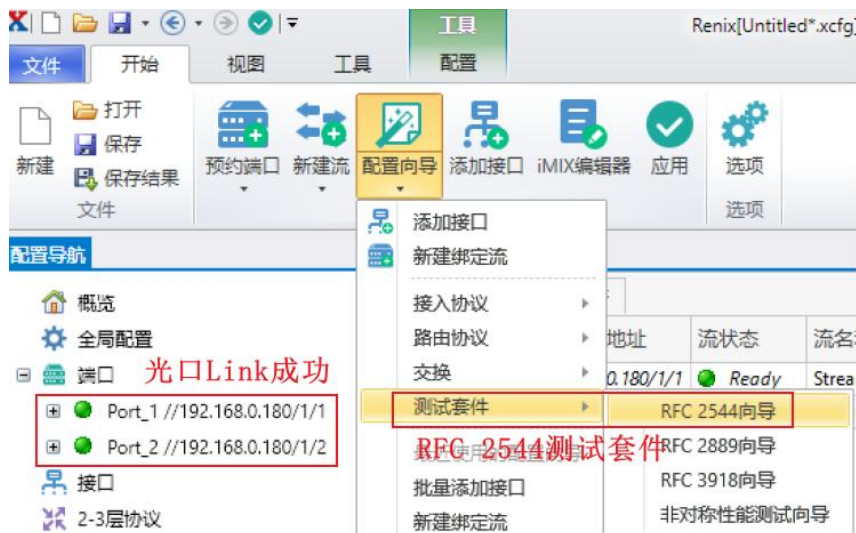


图 2-6 信而泰 Renix 应用程序执行 RFC2544 测试吞吐量测试套件示意图

BigTao220 网络测试仪使用信而泰 Renix 应用程序执行测试功能，执行 RFC2544 测试吞吐量测试套件进行网络性能指标测试^[34]，Spirent TestCenter 网络测试仪测试软件操作类似。在信而泰 Renix 界面功能导航区，选择开始>配置向导>测试套件>RFC2544 向导，选择吞吐量测试测试项，按照向导进行配置，如图 2-6 所示。

配置网络测试仪每轮测试时间 60s,时延类型 FIFO 比特转发(Bit forwarding)模式,流量拓扑点对点(One to One),端点匹配方式轮询调度(Round Robin, RR)。确定最大帧速率负载查找方式为二进制(BINARY)二分查找,设置速率下限为 1%,设置速率上限为 100%,初始速率为 1%,速率精度为 1%,二分法查找百分比为 50%,可接受的丢包率为 0。

2.4.2 基于 Linux 的 TCP/IP 内核协议栈性能测试实验

按照性能测试网络拓扑,将网络设备连接。使用终端 PC 配置攻击检测网关外出方向接口 IP 地址为 192.168.0.1,掩码为 255.255.255.0,网关 IP 地址为 192.168.0.2。进入出方向接口 IP 地址为 192.168.1.1,掩码为 255.255.255.0,网关 IP 地址为 192.168.1.2。配置网络测试仪进入方向接口 IP 地址为 192.168.0.2,掩码为 255.255.255.0,网关 IP 地址为 192.168.0.1。外出方向接口 IP 地址为 192.168.1.2,掩码为 255.255.255.0,网关 IP 地址为 192.168.1.1。在默认情况下, Linux 系统中出于系统安全考虑,系统是禁止的数据包转发的,需手动开启。

表 2-4 基于 Linux 的 TCP/IP 内核协议栈单向转发性能测试汇总表

帧长 <i>L</i> (Bytes)	目标负载 <i>Ot</i> (%)	实际负载 <i>Or</i> (%)	吞吐量 <i>TP</i> (%)	吞吐量 <i>TP</i> (fps)	吞吐量 <i>TP</i> (Mbps)
64	16.469	16.469	16.469	2540744	1646.9
128	29.618	29.618	29.618	2501520	2961.8
256	55.141	55.141	55.141	2497327	5514.1
512	99.227	99.227	99.227	2331461	9922.7
1024	99.227	99.227	99.227	1188062	9922.7
1280	99.227	99.227	99.227	954105	9922.7
1400	99.227	99.227	99.227	873477	9922.7
1518	99.227	99.227	99.227	806461	9922.7

开启 Linux 系统内核协议栈转发模式,终端 PC 使用 SSH 登录待测试的攻击检测网关,执行命令 `echo "1" > /proc/sys/net/ipv4/ip_forward` 即可开启转发服务。使用终端 PC 操作测试仪,配置网络流量数量为一条,方向为单向,执行 RFC2544 吞吐量测试。在基于 Linux 的 TCP/IP 内核协议栈单向转发性能测试中,帧长达到 512 字节以上,平均时延 100-300us,网络目标负载为 99.227%,取整约 100%,网络实际负载为 99.227%,取整约 100%,吞吐量为 9922.7Mbps,即网络转发性达到单向 10G 线速,测试结果如表 2-4 所示。网络中流量通常是

进入和外出同时存在，单向测试结果受限于局部性，再次测试双向转发性能。

表 2-5 基于 Linux 的 TCP/IP 内核协议栈双向转发性能测试汇总表

帧长 L (Bytes)	目标负载 O_t (%)	实际负载 O_r (%)	吞吐量 TP (%)	吞吐量 TP (fps)	吞吐量 TP (Mbps)
64	5.641	5.641	5.641	1678868	1128.201
128	10.282	10.282	10.282	1736824	2056.401
256	19.563	19.563	19.563	1772010	3912.601
512	42.766	42.766	42.766	2009680	8553.201
1024	52.047	52.047	52.047	1246336	10409.401
1280	52.821	52.821	52.821	1015788	10564.201
1400	52.821	52.821	52.821	929946	10564.201
1518	61.328	61.328	61.328	996878	12265.601

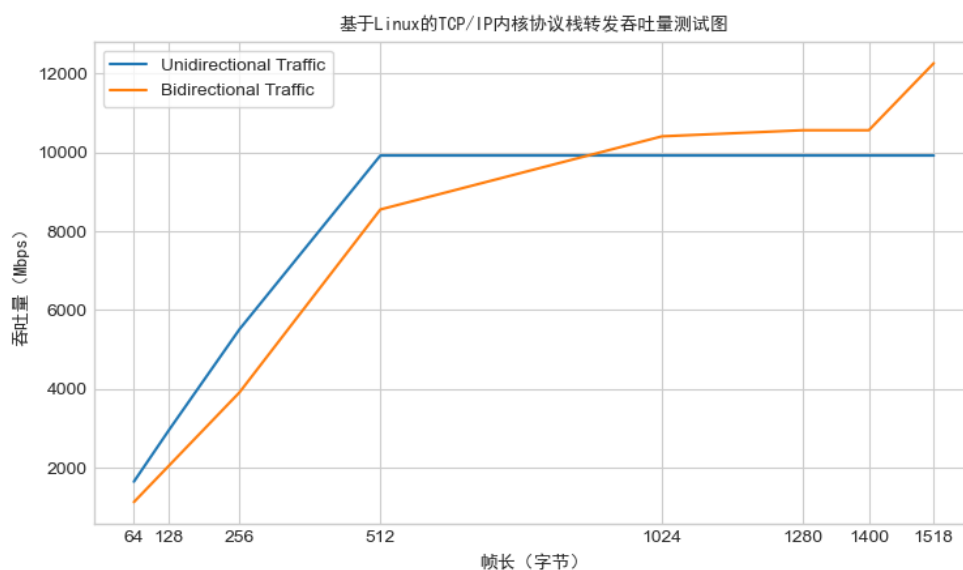


图 2-7 基于 Linux 的 TCP/IP 内核协议栈转发吞吐量测试图

使用终端 PC 操作测试仪，配置网络流量数量为一条，方向改为双向，执行 RFC2544 吞吐量测试。测试结果如表 2-5 所示，基于 Linux 的 TCP/IP 内核协议栈转发吞吐量测试对比如图 2-7 所示。

虽然基于 Linux 的 TCP/IP 内核协议栈单向转发性能测试中 512 字节以上网络转发性达到单向 10G 线速，但是，在基于 Linux 的 TCP/IP 内核协议栈双向转发性能测试中，无论帧长 64 到 1518 字节，网络目标负载与实际负载均无法到达 100%，且帧长 64 到 512 字节间双向转发吞吐量小于单向转发吞吐量，

说明双向网络流量到达内核协议栈后，处理器性能在吞吐量 15000Mbps 左右达到瓶颈，导致超过此吞吐量更多的数据报文到达处理器后丢包。

2.4.3 基于 DPDK 性能测试实验

按照性能测试网络拓扑，将网络设备连接。使用终端 PC 配置网络测试仪进入方向接口 IP 地址和外出方向接口 IP 地址为 192.168.0.2 和 192.168.1.2，配置网络测试仪进入方向网关 IP 地址和外出方向网关 IP 地址为 192.168.0.1 和 192.168.1.1，配置 IP 掩码为 255.255.255.0。

加载 igb_uio.ko 官方网卡驱动，将使用 DPDK 官方 dpdk-devbind.py 工具将两个万兆网卡绑定到 DPDK，测试 DPDK 接管了物理网卡的 I/O 驱动，使用 DPDK 官方 L3fwd 测试程序实现三层网络转发，由于该项测试只有一条流量，所以运行 DPDK-L3fwd 应用程序的核心并不用很多。执行 ./build/l3fwd -l 0-3 -n 4 -- -p 0x3 --config="(0,0,0),(0,1,1),(1,0,2),(1, 1,3)"，开启网络第三层转发服务。使用终端 PC 操作测试仪，配置网络流量数量为一条，方向改为双向，执行 RFC2544 吞吐量测试，测试结果如表 2-6 所示。

表 2-6 DPDK 三层双向转发性能测试汇总表

帧长 <i>L</i> (Bytes)	目标负载 <i>Ot</i> (%)	实际负载 <i>Or</i> (%)	吞吐量 <i>TP</i> (%)	吞吐量 <i>TP</i> (fps)	吞吐量 <i>TP</i> (Mbps)
64	17.243	17.243	17.243	5131844	3448.6
128	31.165	31.165	31.165	5264358	6233
256	58.235	58.235	58.235	5274908	11647
512	100	100	100	4699248	20000
1024	100	100	100	2394636	20000
1280	100	100	100	1923076	20000
1400	100	100	100	1760562	20000
1518	100	100	100	1625486	20000

三层转发实例 L3fwd 基于 DPDK 实现 Internet 的 IPv4、IPv6 数据包路由转发程序，路由匹配模式选择为 LPM。在 DPDK 三层双向转发性能测试中，帧长达到 512 字节以上，平均时延 10-20us，网络目标负载为 100%，网络实际负载为 100%，吞吐量为 20000Mbps，即网络转发性达到双向 10G 线速。

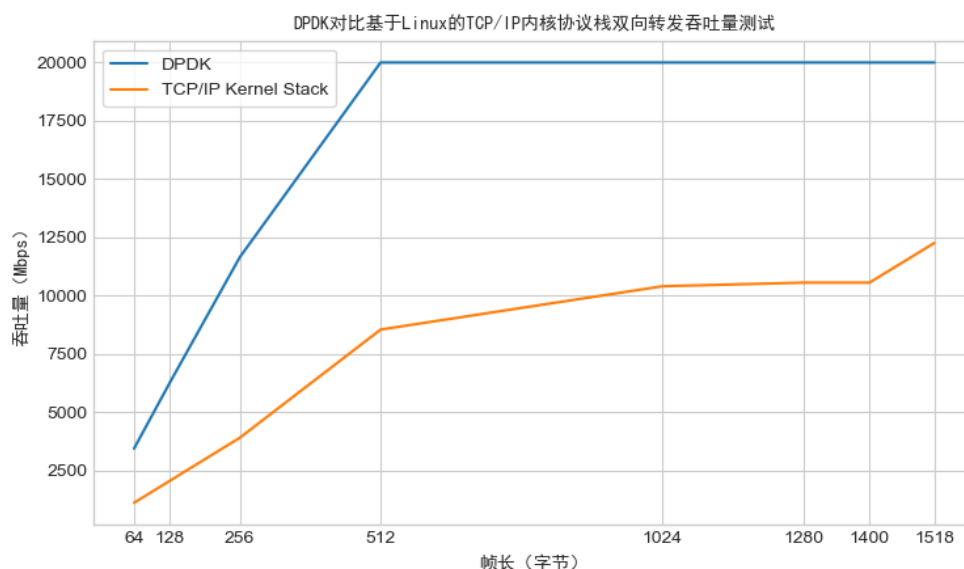


图 2-8 DPDK 对比基于 Linux 的 TCP/IP 内核协议栈双向转发吞吐量测试图

DPDK 对比基于 Linux 的 TCP/IP 内核协议栈双向转发吞吐量测试如图 2-8 所示，可以明显看出无论帧长 64 到 1518 字节，DPDK 双向转发性能都远高于内核协议栈双向转发性能，且 DPDK 在帧长 512 字节后即达到理想负载，证明 DPDK 的性能冗余量充足，证明了基于 DPDK 架构的数据平面作为 APT 攻击检测架构的扩展性和可行性。

2.5 本章小结

本章以课题研究的 APT 攻击检测架构总体设计方案为中心，简要介绍传统网络架构基础和现代 SDN 架构，详细介绍 DPDK 技术特性和网络数据处理的优势，详细介绍 APT 攻击检测总体架构图及各功能模块功能和划分，网络数据旁路内核协议栈，硬件隔离架构保障网络设备不易受到常规网络攻击。突出基于 DPDK 架构的数据平面作为 APT 攻击检测架构的必要性。最后通过实验网络环境，测试同样硬件环境下攻击检测设备，基于传统内核协议栈转发和基于 DPDK 架构转发在网络性能层面的差异。实验结果表明，同样作为软件数据平面技术，基于 DPDK 架构的网络转发性能远远高于基于内核协议栈转发，提高网络数据处理性能从而保障网络设备具备处理网络攻击基础，证明基于 DPDK 架构的数据平面作为 APT 攻击检测架构的可行性。

第3章 基于DPDK的APT攻击流量特性研究

3.1 流表理论基础及定义

3.1.1 SDN 流表基础及定义

在SDN网络架构中，数据平面又被称为转发实体，转发实体执行基于流的转发决策，而不是传统网络执行基于目的节点的转发。转发实体需要具有转发逻辑的流表数据结构，转发实体收到的数据包需要基于流表项内容进行处理，如何提升流表的性能一直是研究的重点问题。

流表分为有状态的流表与无状态的流表两种，状态指数据流的历史统计信息，比如数据流的报文数量，流量信息等。有状态的流表需要对这些信息加以统计，而无状态的流表并不需要。无状态的流表在某些高速网络交换机中，其只负责对数据流匹配转发，并不需要存储流的状态信息，此时应该使用无状态的流表。无状态的流表常被用于二三层交换机，网关等设备中。

有状态的流表被用于QOS，检测DDOS等场景，在该部分场景中，有状态的流表被用于提取数据流特征并进行分析。比如，对数据流加以限速需要统计数据流的流量以此来判断速度，常见的OpenFlow流表就是一种带状态的流表。使用匹配字段匹配数据包流量特征，例如网络数据报五元组特性以太网类型、源MAC地址、目的MAC地址、源IP地址、目的IP地址。

流表是处理与存储数据流信息的二元组列表，其中流标识用于对数据流匹配，具有相同流标识的数据包被视为属于同一条流，通过流标识索引对应的动作指令信息，执行相关动作比如消息加密、端口转发等，每一个流标识与对应的动作指令信息的组合称为一条流表项。

进入的数据包首先在流表中查找，如果到达数据包成功匹配的流表项，转发实体执行对应流表策略操作，统计计数将增加。如果通配符匹配项存在优先级，当多个数据包匹配多个通配符流时，根据优先级从高到低进行匹配。如果数据包无法匹配响应流表项，执行丢弃数据包策略或执行将其上传至上级SDN控制器策略。对于每一个输入数据包，流查找都是必须的，按照网络设备功能、策略可以使用单个流表或多个流表流水线来完成^[35]。

3.1.2 基于TCAM硬件流表

流表通常采用三态内容寻址存储器（Ternary Content Addressable Memory, TCAM）技术实现，TCAM的查找效率与深度无关，无论流表项多大，查找的

速度是一样，即单个时钟周期 $O(1)$ 内提供恒定的流表项查找。网络设备使用流表中填充了一些规则或策略，如 QoS、ACL 和用于快速转发的 IP 路由表。

TCAM 基于内容的地址执行，从而可以快速和并行搜索数据。TCAM 由三个状态组成，通常表示为“0”，“1”和“X”。与传统的二进制存储不同，每个单元只能存储“0”或“1”，TCAM 单元格也可以保持“X”，表示相应的位对于搜索操作而言并不重要。每个 TCAM 单元格通常由多个存储元素，比较器和关联的控制电路组成，以促进基于内容的搜索。执行搜索操作时，TCAM 将输入搜索键与所有单元格中并行的存储值进行比较。TCAM 单元中的“X”值允许更灵活的匹配标准，例如在网络路由和防火墙规则中，其适合通配符或基于范围的搜索常见的应用。TCAM 通常用于路由器和交换机等网络设备中，根据其内容快速识别和路由封装。

相比动态随机存取存储器（Dynamic Random Access Memory, DRAM）和静态随机存取存储器（Static Random-Access Memory, SRAM），随着网络规模扩大到一定程度时，网络设备所需维护的流量状态数量呈指数级增长。以 100 个节点的数据中心网络为例，每秒接收的 TCP 与 UDP 报文数以万计，显著增加了网络设备处理和存储的负载压力。TCAM 功耗高，价格昂贵，并且可用的空间或容量有限，只能容纳 750 到 20000 个流表项。因此，流表相对于所需的规则数来说是比较小的。增加 TCAM 的尺寸会引入其他问题，如成本，并需要高功耗。高速查找时，TCAM 的内存受到几千个表项的限制。SDN 的集中式架构和流表的空间限制还引入了一些性能和安全问题。攻击者可能会通过多次 DoS 攻击来覆盖流表，并操纵控制器拒绝写入合法的流表项。缓解 DoS 攻击需要安装大量的 ACL 分布式规则，而 TCAM 只有有限的容量或空间可用，随着网络规模的扩大，流表的成本将会呈指数增长^[36]。

3.1.3 基于哈希算法软件流表

提高流表性能是提升网络性能的重要措施之一。本章针对硬件流表容量不足的情况下，应用软件流表解决网络攻击状态下硬件流表危机。软件流表通常基于哈希表实现，其中哈希表是流表的核心。

哈希表又称散列表，是根据关键字（key）直接进行访问的数据结构，一般来说，一个关键字（key）对应一个值（value），两者共同组成哈希表的基本单位键值对（entry），存储键值对的数组就叫做哈希表。通过将关键字（key）映射到表中一个位置，可以直接访问记录，以提高查找的速率，相比较其他的查找结构，哈希表查找的时间复杂度为 $O(1)$ ，其查找过程如图 3-1 所示。

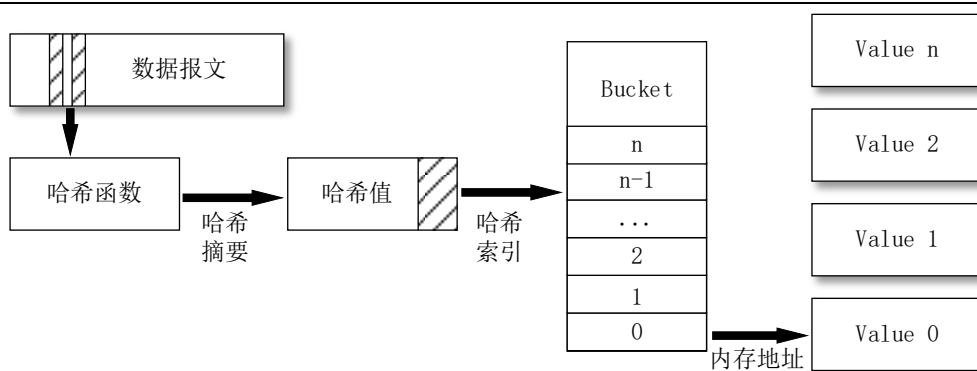


图 3-1 哈希查找示意图

哈希表中用于映射的函数称为哈希函数，哈希函数有多种，常见的哈希函数包括 CRC32, MD5, SHA 等。哈希函数可以将不定长的字符串转化为定长的哈希值。哈希表中的一个存储位置称为一个桶（Bucket），每个桶内可以存储一至多个键值对或键值对对应的索引。哈希表中存储的元素个数与空间大小的比值称为负载因子（Load Factor）。两个不同的 key 通过哈希函数计算的哈希值可能相同，存储这两个 key 时就会发生冲突，称为哈希冲突，哈希冲突的解决方案有多种，称为哈希表算法。

插入，查找与删除为哈希表的基本功能。哈希表性能的评价参数主要包括三个方面，分别为查找速率，插入速率与空间效率。查找速率指在给定一个 key 的情况下，返回查找结果所需要的时间，插入速率指在给定一个 key 的情况下，返回插入结果所需的时间。空间效率指出现必须重哈希或者扩容情景前，最大能插入的 key 的数量与所有尝试进行插入 key 的数量比例。这三个参数是衡量哈希表性能的关键指标。

3.2 基于 DPDK 的二级核间负载均衡软硬协同架构

3.2.1 RSS

随着通信网络技术的发展，而同时网上通信流量也呈线性增长，要求通信具有更小的延迟和更大的吞吐量。但由于网络协议处理过程的影响，系统的应用并不能充分利用物理带宽，并行实现协议的并行处理，网络协议的处理性能就会有很大的提高，对网络速度的影响尤为突出。由于网络流量的压力，基于以前的单核 Linux 内核协议处理模式、通过传统通用处理器单线程运行速度的提升来提高网关的处理速度已经很难满足需求。

RSS^[37]可以实现硬件级别多队列接收网络数据报文，从而实现硬件级多核负载均衡，RSS 的机制在 DPDK 多核架构下，在接收大量网络数据报文时，RSS 将根据哈希算法将这些数据包分配到多个不同的接收队列中，每个队列由一个

处理器核心处理，从而缓解因单个 CPU 过载导致的接收中断处理瓶颈，减少网络延迟，从而提升整体的网络处理性能，RSS 分流机制如图 3-2 所示，原理与哈希查找类似。本文基于申威 3231 多核处理器架构为构建并行处理平台提供了硬件支持，基于此平台，根据 RSS 硬件级别多队列负载均衡的特点并行数据报文处理，提高网关的吞吐率，从而提高网络并行流量处理能力，即提高 APT 攻击流量检查能力。

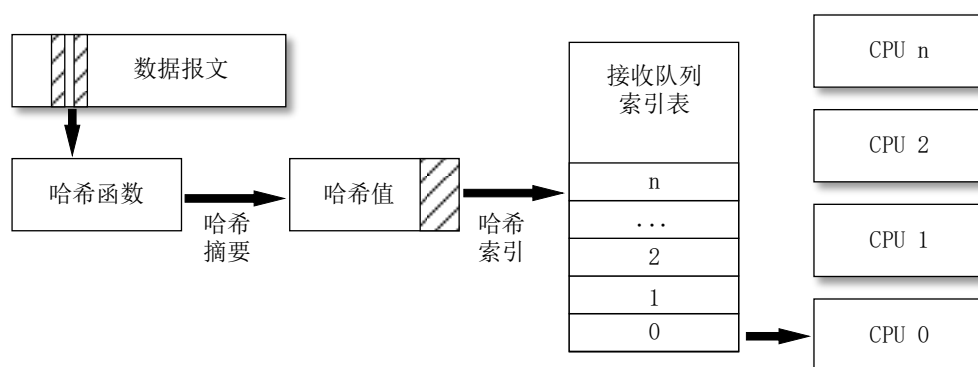


图 3-2 RSS 分流机制图

RSS 实现第一级硬件负载均衡需要接口网卡硬件支持，也需要通过网卡驱动配置 RSS 队列，确保网络数据报文实现正确的到达目标 CPU 核心。在 DPDK 初始化配置网卡时，通常使用 *struct rte_eth_conf* 这种数据结构进行网卡配置，例如下面配置中 *rss_hf = RTE_ETH_RSS_IP*，说明此时的负载均衡方式为依据数据包的源 IP 地址和目的 IP 地址将网络流量分流到不同接收队列。

```
static struct rte_eth_conf port_conf = {
    .rxmode = {
        .mq_mode = RTE_ETH_MQ_RX_RSS,
        .offloads = RTE_ETH_RX_OFFLOAD_CHECKSUM,
    },
    .rx_adv_conf = {
        .rss_conf = {
            .rss_key = NULL,
            .rss_hf = RTE_ETH_RSS_IP,
        },
    },
};
```

根据实际应用通常需要调优 RSS 队列，例如根据流量特征，依据不同的哈希字段合理分流，例如源 IP 地址、目的 IP 地址、源端口号、目的端口号或协议号等。可以设置 RSS 哈希函数参数，针对特定网络报文进行特定网络场景分

流，例如将网络流量完全散列到各个队列，实现 RSS 对称分流。

不同网卡厂商对 RSS 队列及 DPDK 的支持程度不一样，具体需要参考硬件厂商的文档，而不同队列数量跟内存和 CPU 处理能力有关，过多的队列可能会导致数据包在队列中的等待时间，从而加大网络时延，所以并不是队列越多越好，需要根据实际网络运用以及性能测试工具评估最适合的 RSS 参数。

DPDK 中使用的默认哈希密钥（也是 Microsoft 推荐的密钥）不会将对称流均分到 CPU。Intel 82599 网卡的文档中，RSS 哈希函数可以采用微软的托普利兹算法（Toeplitz Hash），在计算过程中使用到一个 320 位（40 字节）的 random secret key 作为加密密钥，实现网络流量的对称分流。例如 Intel 82599 网卡中在 RSS Random Key Register (RSSRK) 寄存器保存这个 Key 作为参数，对于 DPDK 架构则需要将 *struct rte_eth_conf* 中 *rss_key* 进行修改配置。Toeplitz Hash 算法及 Intel 82599 网卡对称 *rss_key*，即 Toeplitz 矩阵 RSK (Receive Side Key) 参数共 320 位（40 字节），如下算法 3-1 所示。

算法 3-1. Toeplitz Hash 算法

输入: *input*;

输出: *digest*.

1.初始化 *digest*;

2.Intel 82599 网卡对称 *RSK[40]* =

```
0x6d 0x5a 0x6d 0x5a 0x6d 0x5a 0x6d 0x5a 0x6d 0x5a
0x6d 0x5a 0x6d 0x5a 0x6d 0x5a 0x6d 0x5a 0x6d 0x5a
0x6d 0x5a 0x6d 0x5a 0x6d 0x5a 0x6d 0x5a 0x6d 0x5a
0x6d 0x5a 0x6d 0x5a 0x6d 0x5a 0x6d 0x5a 0x6d 0x5a
```

3.FOREACH *bit* in *input*

4. IF *bit* == 1 THEN *digest* = *digest* 异或 *RSK*(左 32bit);

5. 左移位 *RSK* << 1;

6.RETURN *digest*.

3.2.2 布谷鸟哈希

在网络设备中，查找是其核心功能，稳定性是其核心诉求。而传统的哈希算法难以均衡高负载下的稳定性与查找性能。如何在满足高空间效率的同时，均衡哈希表的查找与插入性能一直是业界关注的核心问题。

布谷鸟哈希最早由 Rasmus Pagh 和 Flemming Friche Rodler 在 2001 年提出。布谷鸟哈希是为了解决哈希冲突问题的方案之一，原理是利用空间换时间的方法。名称源于仿生学现象类似于布谷鸟将蛋下在其他种族的鸟巢中，布谷幼鸟

并将其他种族的幼鸟或鸟蛋挤出巢穴的行为。布谷鸟哈希具有占用内存小、查找效率高等特性，通常用于内存管理和 Bloom 过滤器。通过基于布谷鸟哈希表的流表实现第二级软件负载均衡，布谷鸟哈希表的性能直接决定了网络的性能。

布谷鸟哈希表是一种高效的哈希表实现，可以支持单个数据的插入和查询操作，布谷鸟哈希表可以由单哈希表组成，如图 3-3 所示。每个子表由 Bucket 组成并且每个 Bucket 有若干个内存地址单元。通常哈希表使用 2 个 Hash 函数 h_1 、 h_2 ，每个 Hash 函数都会将插入的元素均匀地映射到对应 Hash 表中的一个桶内。给定元素 x ，它只会被放置在子表中的某一个，即在子表的 $h_1(x)$ 桶内。同时，由于一个哈希桶内最多有一个内存单元用于存储元素，因此，当有超过 2 个元素映射到同一个桶内同一个内存单元，需要将冲突的元素移动到其他子表内的桶中。

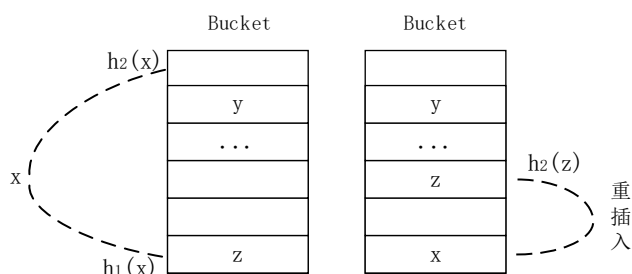


图 3-3 布谷鸟哈希单表插入及重插入示意图

如图 3-3 中所示， x 元素在子表内映射 $h_1(x)$ 的对应内存单元位置都已经被占用了，因此需要将子表中第一个内存单元内冲突的元素 z 先踢出子表，然后重新计算 $h_2(z)$ 内存地址单元并插入到子表。这里如果下一个内存地址单元还是被占用，上述过程要重复多次直到有一个空位置被找到。如果重复多次后依旧没有空位置，那就表明当前哈希表已经装满，需要重新扩容以容纳更多的元素。

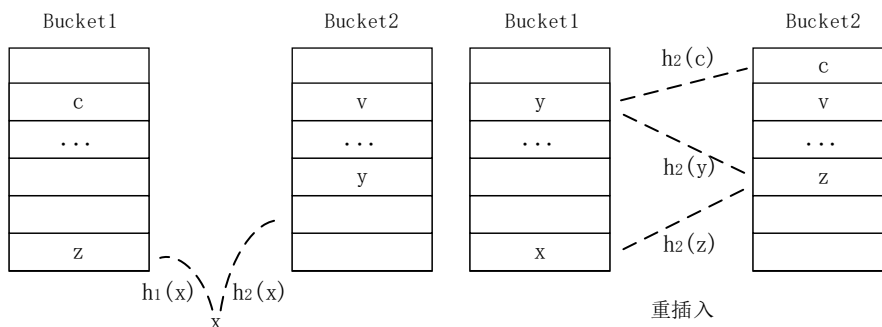


图 3-4 布谷鸟哈希多表插入及重插入示意图

如图 3-4 所示，布谷鸟哈希表也可以由多哈希表组成。与单哈希表不同的是， x 元素在第一个子表内映射 $h_1(x)$ 的对应内存单元位置都已经被占用了，因此需要将子表中第一个内存单元内冲突的元素 z 先踢出子表，然后重新计算 $h_2(z)$ 内存地址单元并插入到第二个子表。如果下一个内存地址单元还是被占用，与

单哈希表类似上述过程要重复多次直到空位置被找到。

本文所述流表使用多哈希表存储，在哈希表中的所有 *key* 的内存地址单元及其与每个 *key* 相关联的数据均存储到相应表项中。使用哈希函数将输入 *key* 转换为 4 字节 *key* 签名。对于大的 *key*，将输入 *key* 与来自存储桶的 *key* 进行比较比将输入 *key* 的 4 字节签名与来自存储桶的 *key* 签名进行比较要花费更多的时间。因此，首先完成签名比较，仅在签名匹配时才完成 *key* 比较。桶索引值是将哈希 *key* 签名对哈希桶数取模数的值。对于任何输入 *key*，有两个可能的桶（主要桶和次要桶），其中该 *key* 可以存储在散列中，因此只有当查询 *key* 时才需要检查桶中的条目。与通过线性扫描阵列中的所有条目的基本方法相反，通过将散列条目的总数减少到两个哈希桶中的条目数来减少要扫描的条目数以提升查找速度。

但是，执行查找操作存在三种可能性，目标元素位于 $h_1(x)$ 内存地址单元，位于 $h_2(x)$ 内存地址单元，或者目标元素不存在，但引入分支的话会让代码不能向量化执行，工程上可以采用掩码（mask）的技巧消除分支，对于向量化执行查找操作可以避免多线程碰撞问题。这里提供一种基于布谷鸟哈希表多线程并行执行哈希查找操作的算法 3-2。

算法 3-2.布谷鸟哈希表多线程并行执行哈希查找操作

输入：*key* 值；

输出：成功返回内存地址，失败返回 NULL.

- 1.初始化局部变量 *address*, *mask1* 和 *mask2*;
 - 2.计算哈希值，即 $h_1(x)$ 和 $h_2(x)$ 映射到哈希桶的内存位置单元；
 - 3.索引 $h_1(x)$ 和 $h_2(x)$ 哈希桶的内存位置单元存储的 *key* 值与输入 *key* 值 *x* 判断两个值大小是否相同，判断后值取反即为对应掩码 *mask1* 和 *mask2*;
 - 4.实际存储内存地址单元 *address* 即 $h_1(x)$ 与 *mask1* 或 $h_2(x)$ 与 *mask2*;
 - 5.**RETURN** 实际存储内存地址单元 *address*.
-

同样，对于多核处理器环境，需要探索如何并行执行哈希计算和插入操作。常规情况下需要在执行插入操作时加锁，保证数据的一致性。在多线程环境中使用，需要实现线程安全，每个线程单独计算和插入该线程维护的哈希桶。使用 CAS（Compare And Swap）硬件指令实现的原子操作，在执行完毕之前不会被任何其它进程或事件中断，实际可以通过硬件指令、编译器或操作系统提供的同步原语来实现。在不使用锁的情况下实现多线程并发编程中的线程安全，避免多线程内存地址段错误发生。提供一种基于布谷鸟哈希表多线程并行执行哈希计算和插入操作的算法 3-3。

算法 3-3.布谷鸟哈希表多线程并行执行哈希计算和插入操作

输入：待插入元素 x ；

输出：是否插入成功

- 1.初始化局部变量 i ；
- 2.计算哈希值，即 $h_1(x)$ 和 $h_2(x)$ 映射到哈希桶的内存位置单元；
- 3.WHILE 哈希桶的内存位置单元已存在元素 E_i
4. 新元素 x 替换插入（CAS 原子操作） E_i 内存位置单元；
5. $x = E_i$ 旧元素重新计算 $h_1(x)$ 和 $h_2(x)$ 内存位置单元；
6. IF i 大于哈希桶最大存放数量
7. RETURN 元素插入失败，重新扩容以容纳更多的元素.
- 8.RETURN 元素插入成功.

3.3 二级核间负载均衡软硬协同架构性能测试实验

3.3.1 哈希算法性能仿真测试实验环境搭建

选择合适的 Hash 函数是影响整体性能的关键因素之一。不同的 Hash 函数可能导致不同的冲突率和插入效率。

表 3-1 硬件运行环境与测试单元的关系

CPU	DDR4	操作系统	测试单元
Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz	16GB	Ubuntu 22.04.4 LTS	Hash 性能

表 3-2 软件运行环境与功能的关系

软件运行环境/依赖	功能	软件/内核版本
GCC	编译工具	11.4.0
Python	脚本工具	3.10.12
glibc	运行库	Ubuntu GLIBC 2.35
uthash	Hash 算法	https://github.com/troydhanson/uthash/ 2.3.0
DPDK	Cockoo Hash 算法	24.03.0-rc1
Ubuntu 22.04.4 LTS	操作系统	Linux 5.15.146.1 #1 SMP x86_64 GNU/Linux

由于对比不同 Hash 算法间性能差别，实验环境采用同硬件运行环境同操作系统，测出实验数据，硬件运行环境见表 3-1。仿真实验采用八种 Hash 算

法，其中六种 Hash 算法使用 uthash 开源库中 Fowler-Noll-Vo Hash 算法、Jenkins Hash 算法、Paul Hsieh Hash 算法、OAT Hash 算法、Bernstein Hash 算法和 SAX Hash 算法，剩下两种 Hash 算法使用 DPDK 算法库中的基于 Cockoo Hash 实现的 CRC Hash 算法和 JHash 算法，软件运行环境见表 3-2。

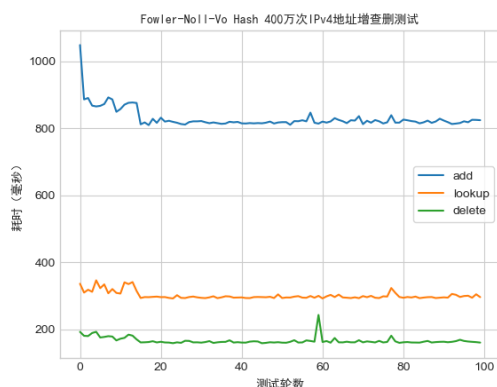
3.3.2 哈希算法性能仿真测试实验

为了更好地模拟逼近 APT 攻击流量特征，本文采用了 CAIDA (The Center for Applied Internet Data Analysis) 数据集、加拿大网络安全研究所 (CIC) 发布 CIC-IDS2018 数据集和 CIC-IDS2017-PCAP 数据集进行测试，CAIDA 数据集是网络流量分析数据集，从骨干网络流量中收集的真实数据，包含 700 万个不同的元素。CIC-IDS2018 数据集用于检测网络攻击和异常行为，数据集包括多种类型的攻击，如 DDoS、Web 攻击、渗透测试等。CIC-IDS2017-PCAP 数据集目前已成为网络入侵检测研究中的标准数据集之一，不仅被广泛应用于学术研究，还被工业界用于开发和测试新的入侵检测系统。本章从上述实验数据集中随机选取了包含 1000 条到 400 万条不同网络流量，用于进行测试，目标数据类型选取目的 IPv4 作为 Key，采用实验数据集信息见表 3-3 所示。

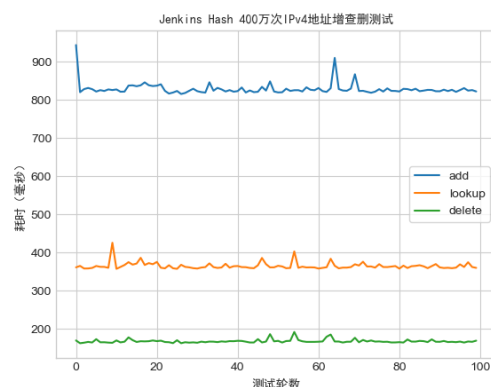
表 3-3 实验数据集信息

数据集	记录条数	目标数据类型
CAIDA	约 700 万条	目的 IPv4
CIC-IDS2018	约 280 万条	目的 IPv4
CIC-IDS2017-PCAP	约 120 万条	目的 IPv4

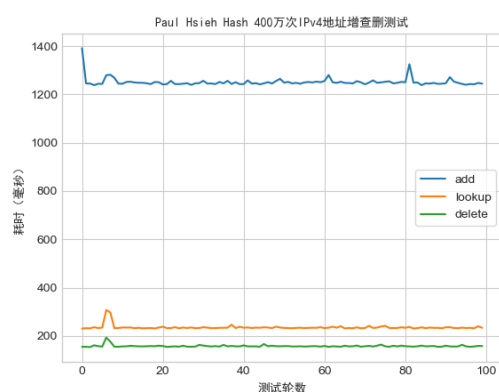
(1) 不同算法 400 万次添加、查找、删除的实验数据，见图 3-5。



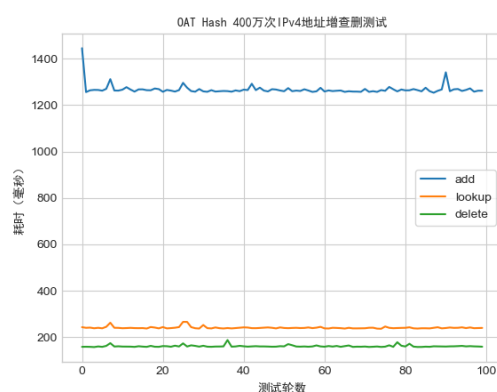
a) Fowler-Noll-Vo Hash 算法 400 万次 IPv4 增\查\删性能测试图



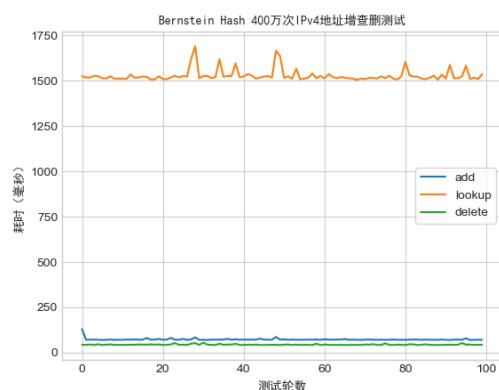
b) Jenkins Hash 算法 400 万次 IPv4 增\查\删性能测试图



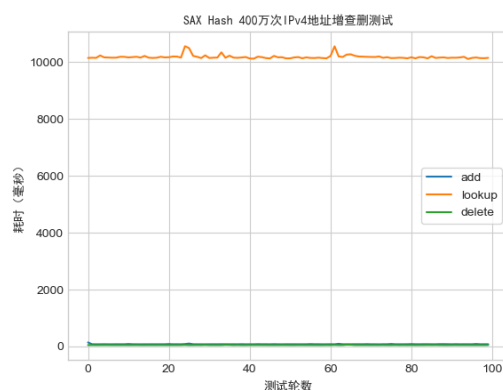
c) Paul Hsieh Hash 算法 400 万次 IPv4 增/查/删性能测试图



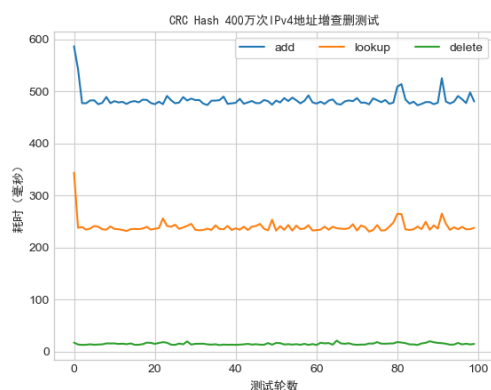
d) OAT Hash 算法 400 万次 IPv4 增/查/删性能测试图



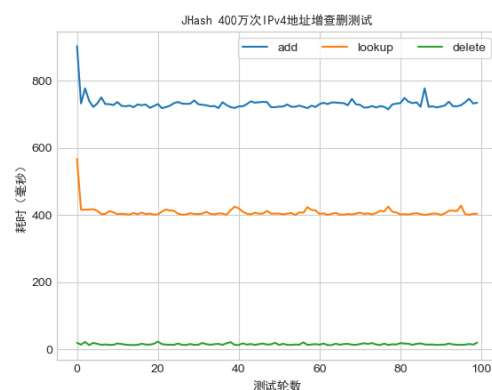
e) Bernstein Hash 算法 400 万次 IPv4 增/查/删性能测试图



f) SAX Hash 算法 400 万次 IPv4 增/查/删性能测试图



g) CRC Hash 算法 400 万次 IPv4 增/查/删性能测试图



h) JHash 算法 400 万次 IPv4 增/查/删性能测试图

图 3-5 不同 Hash 算法 400 万次 IPv4 地址增/查/删性能对比图

如图 3-5(f)所示, 不同 Hash 算法 400 万次 IPv4 地址增/查/删性能对比测试中, 每种算法增/查/删 400 万键值对。其中 SAX Hash 算法的特性是添加、删除耗时接近于 0, 但查找耗时非常高, 显然该算法并不适用于查找高密集型的场

景使用。如图 3-5(e)所示, Bernstein Hash 算法与 SAX Hash 算法特性类似, 虽然查找性能相较 SAX Hash 算法提升 90%左右, 但较高的查找耗时依然不适用于查找密集型的场景使用。根据 Hash 算法查找密集型的需求, 每种算法使用 400 万组 IPv4 地址作为 Key, 添加 400 万键值对, 并针对不同 Hash 算法 400 万次 IPv4 地址查找性能对比。

(2) 不同 Hash 算法 400 万次 IPv4 地址查找性能对比数据, 见图 3-6。

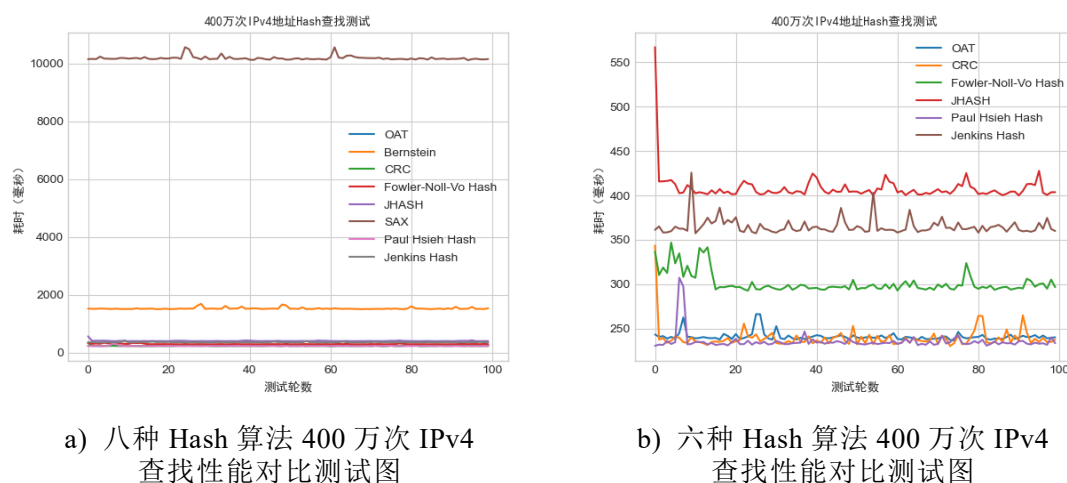


图 3-6 不同 Hash 算法 400 万次 IPv4 地址查找性能对比图

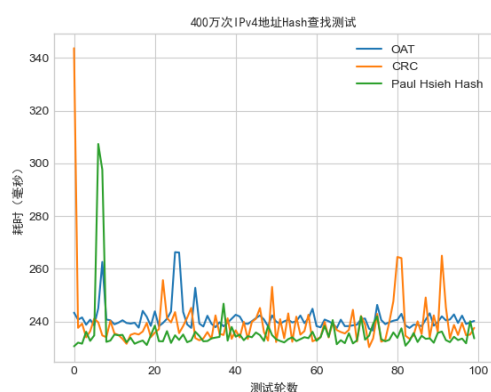
如图 3-6(a)所示, 测试八种 Hash 算法 400 万次 IPv4 键值对查找中, SAX Hash 算法查找耗时 10s 左右, Bernstein Hash 算法查找耗时 1.8s 左右, 相较其余 Hash 算法耗时 400ms 内, 显而易见, SAX Hash 算法和 Bernstein Hash 算法不能在查找密集型的场景使用, 导致测试结果不具有可观察性。去除 SAX Hash 算法和 Bernstein Hash 算法影响后, 对剩下六种 Hash 算法进行对比测试。

(3) 剩下六种 Hash 算法 IPv4 地址查找性能对比数据, 见图 3-6(b)。

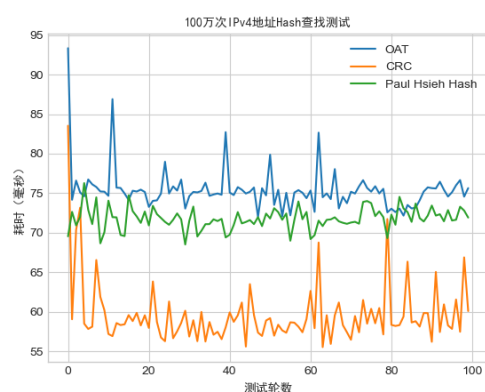
每种算法使用 400 万组 IPv4 地址作为 Key, 并增加 400 万键值对, 测试剩下六种 Hash 算法 400 万次 IPv4 键值对查找中, 取算法稳定运行区间耗时, 采用基于 Cockoo Hash 的 JHash 算法查找耗时 400ms 左右, Jenkins Hash 算法查找耗时 360ms 左右, Fowler-Noll-Vo Hash 算法查找耗时 300ms 左右, 其中 Fowler-Noll-Vo Hash 算法稳定运行后, 执行耗时更低、稳定性更高, 但相较剩余三种 Hash 算法耗时依然较高。去除 Jhash 算法、Jenkins Hash 算法和 Fowler-Noll-Vo Hash 算法后, 针对剩下三种 OAT/CRC/Paul Hsieh Hash 算法进行对比测试。

(4) 剩下三种 Hash 算法 IPv4 地址查找性能对比数据, 见图 3-7。

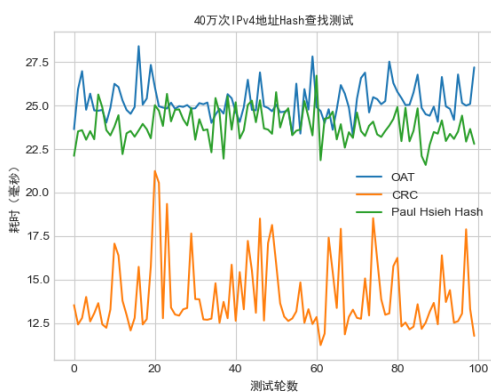
每种算法使用 400 万组 IPv4 地址作为 Key, 分别添加不同数量级键值对 (400 万键值对、100 万键值对、40 万键值对、10 万键值对、1 万键值对和 1000 键值对), 针对键值对查找性能, 再进行不同数量级查找对比测试。



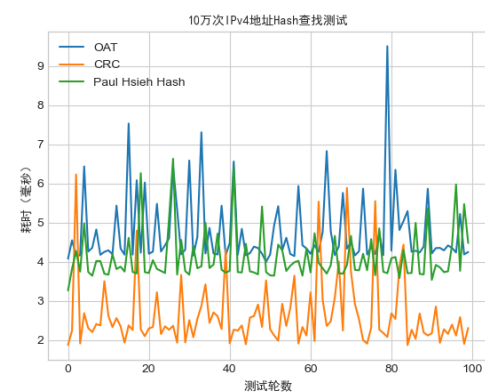
a) OAT/CRC/Paul Hsieh Hash 算法
400 万次 IPv4 查找性能对比测试图



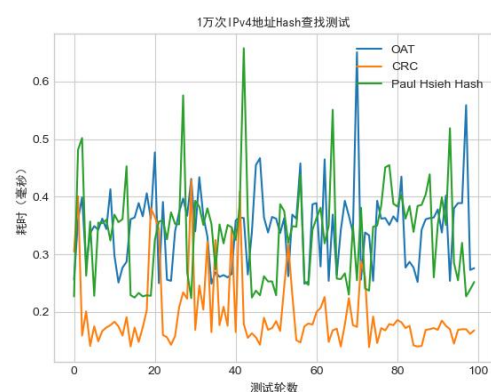
b) OAT/CRC/Paul Hsieh Hash 算法
100 万次 IPv4 查找性能对比测试图



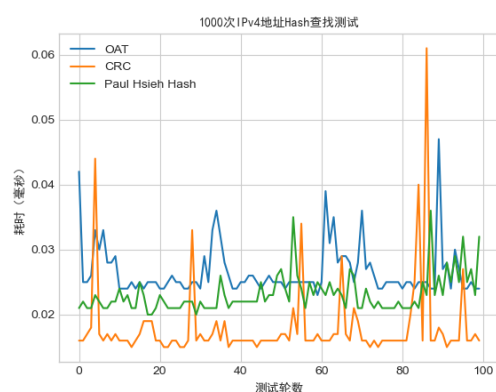
c) OAT/CRC/Paul Hsieh Hash 算法
40 万次 IPv4 查找性能对比测试图



d) OAT/CRC/Paul Hsieh Hash 算法
10 万次 IPv4 查找性能对比测试图



e) OAT/CRC/Paul Hsieh Hash 算法
1 万次 IPv4 查找性能对比测试图



f) OAT/CRC/Paul Hsieh Hash 算法
1000 次 IPv4 查找性能对比测试图

图 3-7 OAT/CRC/Paul Hsieh Hash 算法 IPv4 地址查找性能对比图

如图 3-7 所示，在 100 万次、40 万次、10 万次、1 万次和 1000 次的查找测试中基于的 CRC 算法的查找计算耗时上都明显优于其他算法，但由于 CRC 算法的散列性较其余 Hash 算法不足，导致存在碰撞较多情况算法查找耗时波

动性较大。相较三种算法，Paul Hsieh Hash 算法查找性能适中，查找稳定性较高，且在 400 万大量键值对情况下，依然保持稳定性能。

结合基于 Cockoo Hash 的 CRC 算法的计算速度上的优势，以及 Paul Hsieh Hash 算法查找性能适中的特性，将 Paul Hsieh Hash 算法融入 Cockoo Hash 算法理论结构中，优化后算法命名为基于 Cockoo Hash 的 Paul Hsieh Hash 算法，简称 Cockoo Paul Hsieh Hash。针对基于 Cockoo Hash 的 CRC 算法，Paul Hsieh Hash 算法和融合优化后的基于 Cockoo Hash 的 Paul Hsieh Hash 算法对比测试。

(5) 优化后三种 Hash 算法 IPv4 地址查找性能对比数据，见图 3-8/3-9。

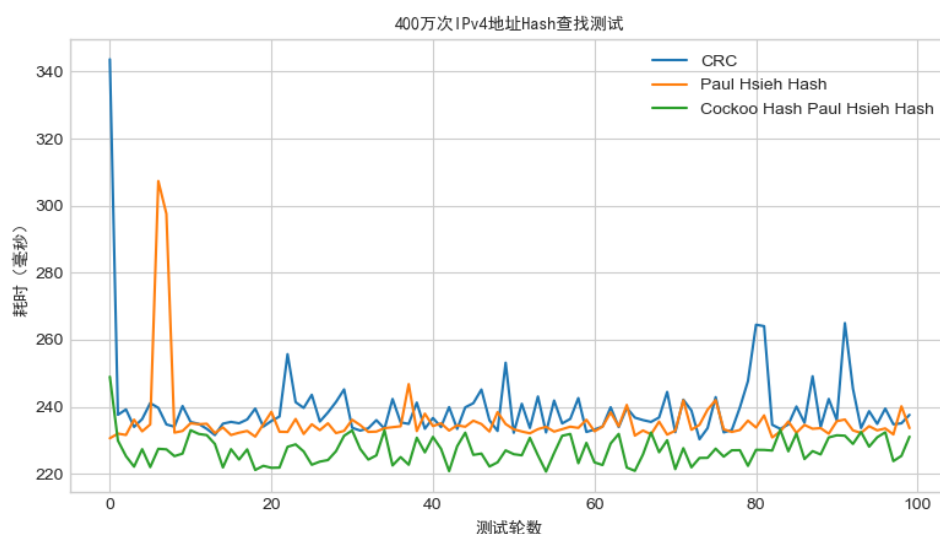


图 3-8 CRC/Paul Hsieh Hash/Cockoo Paul Hsieh Hash 算法 IPv4 地址 400 万次查找对比图

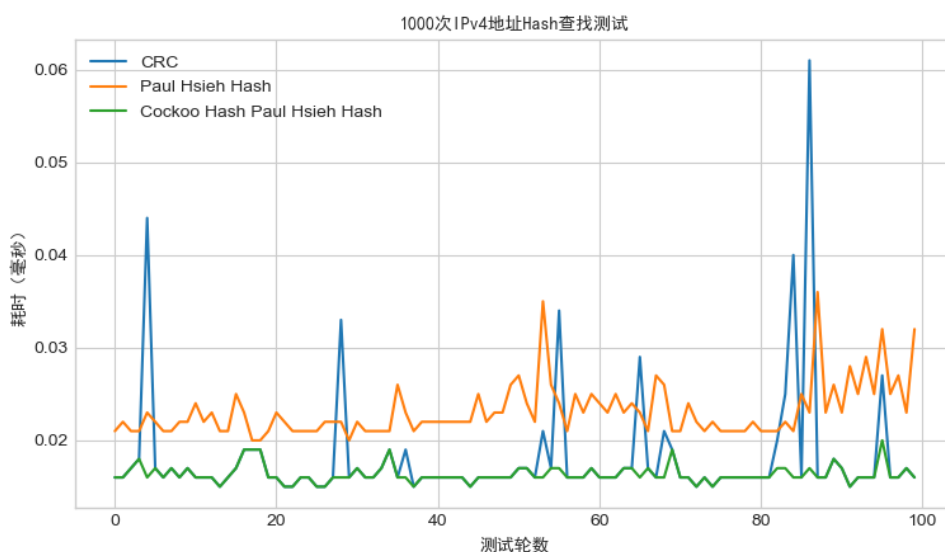


图 3-9 CRC/Paul Hsieh Hash/Cockoo Paul Hsieh Hash 算法 IPv4 地址 1000 次查找对比图

如图 3-8、图 3-9 所示，CRC/Paul Hsieh Hash/Cockoo Paul Hsieh Hash 算法，

粒度采用最大使用 400 万组 IPv4 地址作为 Key，最小使用 1000 组 IPv4 地址作为 Key。在添加 400 万键值对后，测试 400 万次 IPv4 键值对查找中，基于优化后的 Cockoo Hash 的 Paul Hsieh Hash 算法查找耗时 220ms-230ms 之间，整体执行耗时更低、稳定性更高。在使用 1000 组 IPv4 地址作为 Key，并添加 1000 键值对，测试 1000 次 IPv4 键值对查找中，基于优化后的 Cockoo Hash 的 Paul Hsieh Hash 算法查找性能曲线波动更小。

即基于 Cockoo Hash 优化改造后的 Paul Hsieh Hash 算法在 APT 网络攻击环境中，依据流量特性执行流表查找操作时，查找效率、查找稳定性均优于本文所述其余 Hash 算法，即均优于本文所述其余 Hash 算法构成的 Hash 流表。

3.3.3 RSS 压力测试实验环境搭建

硬件运行环境见表 3-4，实验环境使用与第 2 章性能测试实验相同硬件运行环境，相同操作系统，由于仅测试 RSS 分流能力，不需要精确测出网络吞吐量，可以在发送方 PC 和接收方 PC 均安装 iperf 网络性能测试软件。

表 3-4 硬件运行环境与测试单元的关系

CPU	DDR4	操作系统	测试单元
国产申威 3231CPU	16GB	统信 UOS	RSS 压力测试（被测）
Intel(R) Core(TM) i5-8250U	16GB	Ubuntu 22.04.4 LTS	RSS 压力测试
CPU @ 1.60GHz			（测试工具 iperf 3.9）

表 3-5 软件运行环境与功能的关系

软件运行环境/依赖	功能	软件/内核版本
GCC	编译工具	8.3.0
Python	脚本工具	3.10.12
DPDK	数据平面	19.11
L2fwd 转发实例	二层网络转发软件	DPDK-19.11
iperf	网络性能测试软件	3.9 (cJSON 1.7.13)
统信 UOS V18	操作系统（被测）	Linux 4.19.0-sw64-server #225 SMP sw64 GNU/Linux
Ubuntu 22.04.4 LTS	操作系统（PC）	Linux 5.15.146.1 #1 SMP x86_64 GNU/Linux

DPDK 相关软件使用性能测试实验，相同软件运行环境，相同系统参数，同样统信 UOS 操作系统，加载官方 igb_uio.ko 驱动，软件运行环境见表 3-5。

iperf 网络性能测试软件运行在 PC 上，由于受硬件运行环境制约，只能可

以粗略测试最大 TCP 和 UDP 带宽性能，满足测试项多核负载均衡测试需求，iperf 具有多种参数和 UDP 特性，可以根据需要调整，可以报告带宽、延迟抖动和数据包丢失。由客户端 iperf 产生流量，通过攻击检测网关二层转发给接收方 PC 服务端 iperf，网络测试拓扑如图 3-10 所示。

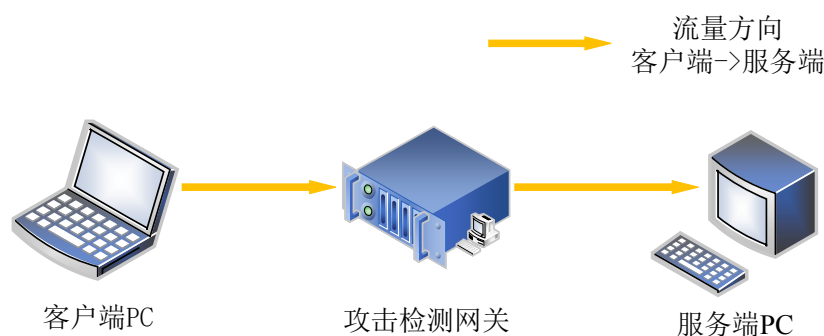


图 3-10 RSS 压力测试网络拓扑

3.3.4 RSS 压力测试实验

按照 RSS 压力测试网络拓扑，将网络设备连接。加载 DPDK 官方 igb_uio.ko 驱动，将使用 DPDK 官方 dpdk-devbind.py 工具将两个万兆网卡绑定到 DPDK，测试 DPDK 接管了物理网卡的 I/O 驱动，使用 DPDK 官方 L2fwd 测试程序实现二层网络转发，L2fwd 基于 DPDK 实现一个简单的网络交换程序。由于该项测试需要查看每个 CPU 核心收到报文的数量，所以运行 DPDK-L2fwd 应用程序需要更多核心。

(1) DPDK-L2fwd 应用程序非 RSS 模式下 执行 ./build/l2fwd -l 0-15 -n 4 -p 0x3 --config="(0,0,0),(0,1,1),(0,2,2),(0,3,3),(0,4,4),(0,5,5),(0,6,6),(0,7,7),(1,0,8),(1,1,9),(1,2,10),(1,3,11),(1,4,12),(1,5,13),(1,6,14),(1,7,15)"，参数表示使用 16 个处理线程分别运行在 16 个 CPU 核心中，开启网络第二层转发服务。

表 3-6 非 RSS 模式下 CPU 核心收到报文数量汇总表

CPU 0	CPU 1	CPU 2	CPU 3	CPU 4	CPU 5	CPU 6	CPU 7
P (Bytes)	P (Bytes)	P (Bytes)	P (Bytes)	P (Bytes)	P (Bytes)	P (Bytes)	P (Bytes)
0	0	0	0	0	0	0	0

表 3-6 (续表)

CPU 8	CPU 9	CPU 10	CPU 11	CPU 12	CPU 13	CPU 14	CPU 15
P (Bytes)	P (Bytes)	P (Bytes)	P (Bytes)	P (Bytes)	P (Bytes)	P (Bytes)	P (Bytes)
11.6G	0	0	0	0	0	0	0

(2) DPDK-L2fwd 应用程序 RSS 模式下 通过修改 DPDK-L2fwd 代码中的 rss_key 配置为上文中所述的对称 rss_key，重新编译后执行 ./build/l2fwd -l 0-15

-n 4 -- -p 0x3 --config="(0,0,0),(0,1,1),(0,2,2),(0,3,3),(0,4,4),(0,5,5),(0,6,6),(0,7,7),(1,0,8),(1,1,9),(1,2,10),(1,3,11),(1,4,12),(1,5,13),(1,6,14),(1,7,15)", 参数表示使用 16 个处理线程分别运行在 16 个 CPU 核心中, 开启网络第二层转发服务。

同上所述, 配置服务端 PC 的 IP 地址、掩码, 配置客户端 PC 的 IP 地址、掩码。服务端 PC 执行命令 iperf3 -s 运行 iperf 服务端, 如图 3-11 所示。客户端 PC 执行命令 iperf3 -c 192.10.0.1 -b 10G, 表示客户端向 192.10.0.1 以 10Gbits/s 的速率向服务端发送流量, 如图 3-12 所示。

```
root@t480s:/home/user# iperf3 -s
-----
Server listening on 5201
-----
Accepted connection from 192.10.0.1, port 55614
[ 5] local 192.10.0.1 port 5201 connected to 192.10.0.1 port 55616
[ ID] Interval            Transfer        Bitrate
[ 5] 0.00-1.00      sec  1.11 GBytes    9.51 Gbits/sec
[ 5] 1.00-2.00      sec  1.16 GBytes    10.0 Gbits/sec
[ 5] 2.00-3.00      sec  1.16 GBytes    10.0 Gbits/sec
[ 5] 3.00-4.00      sec  1.16 GBytes    10.0 Gbits/sec
[ 5] 4.00-5.00      sec  1.16 GBytes    10.0 Gbits/sec
[ 5] 5.00-6.00      sec  1.16 GBytes    9.99 Gbits/sec
[ 5] 6.00-7.00      sec  1.16 GBytes    10.0 Gbits/sec
[ 5] 7.00-8.00      sec  1.16 GBytes    10.0 Gbits/sec
[ 5] 8.00-9.00      sec  1.16 GBytes    10.0 Gbits/sec
[ 5] 9.00-10.00     sec  1.16 GBytes    10.0 Gbits/sec
[ 5] 10.00-10.05    sec  58.0 MBytes    10.0 Gbits/sec
-----
[ ID] Interval            Transfer        Bitrate
[ 5] 0.00-10.05     sec  11.6 GBytes    9.95 Gbits/sec
-----
Server listening on 5201
-----
```

图 3-11 服务端 PC 运行 iperf3 接收流量示意图

```
root@t480s:/home/user# iperf3 -c 192.10.0.1 -b 10G
Connecting to host 192.10.0.1, port 5201
[ 5] local 192.10.0.1 port 39394 connected to 192.10.0.1 port 5201
[ ID] Interval            Transfer        Bitrate      Retr  Cwnd
[ 5] 0.00-1.00      sec  1.16 GBytes    9.99 Gbits/sec    0   1.69 MBytes
[ 5] 1.00-2.00      sec  1.16 GBytes    10.0 Gbits/sec    0   1.69 MBytes
[ 5] 2.00-3.00      sec  1.16 GBytes    10.0 Gbits/sec    0   1.69 MBytes
[ 5] 3.00-4.00      sec  1.16 GBytes    10.0 Gbits/sec    0   1.69 MBytes
[ 5] 4.00-5.00      sec  1.16 GBytes    10.0 Gbits/sec    0   1.69 MBytes
[ 5] 5.00-6.00      sec  1.16 GBytes    10.0 Gbits/sec    0   1.69 MBytes
[ 5] 6.00-7.00      sec  1.16 GBytes    10.0 Gbits/sec    0   1.69 MBytes
[ 5] 7.00-8.00      sec  1.16 GBytes    10.0 Gbits/sec    0   1.69 MBytes
[ 5] 8.00-9.00      sec  1.16 GBytes    10.0 Gbits/sec    0   1.69 MBytes
[ 5] 9.00-10.00     sec  1.16 GBytes    10.0 Gbits/sec    0   1.69 MBytes
-----
[ ID] Interval            Transfer        Bitrate      Retr
[ 5] 0.00-10.00     sec  11.6 GBytes    10.0 Gbits/sec    0
[ 5] 0.00-10.05     sec  11.6 GBytes    9.95 Gbits/sec
-----
iperf Done.
```

图 3-12 客户端 PC 运行 iperf3 发送流量示意图

同上所述, 配置服务端 PC 的 IP 地址、掩码, 配置客户端 PC 的 IP 地址、掩码。服务端 PC 执行命令 iperf3 -s 运行 iperf 服务端, 如图 3-11 所示。客户端 PC 执行命令 iperf3 -c 192.10.0.1 -b 10G, 表示客户端向 192.10.0.1 以 10Gbits/s 的速率向服务端发送流量, 如图 3-12 所示。

通过串口打印查看 DPDK-L2fwd 应用程序 16 个 CPU 核心收到的报文统计, 各个核心收到报文数量汇总如表 3-7 所示。

表 3-7 RSS 模式下 CPU 核心收到报文数量汇总表

CPU 0	CPU 1	CPU 2	CPU 3	CPU 4	CPU 5	CPU 6	CPU 7
<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)
725M	725M	725M	725M	725M	725M	725M	725M

表 3-7 (续表)

CPU 8	CPU 9	CPU 10	CPU 11	CPU 12	CPU 13	CPU 14	CPU 15
<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)	<i>P</i> (Bytes)
725M	725M	725M	725M	725M	725M	725M	725M

在 RSS 负载环境下, 在单一流量环境中, RSS 模式网卡将利用多队列将报文对称分流到所有的核心, 所有核心均匀收到报文后, 最大程度利用多核处理器并行计算优势, 网络处理性能得到很大的提高。

3.4 本章小结

本章以课题研究的 APT 攻击的流量特征为中心, 提高 APT 攻击流量检查能力就是需要提高流表性能, 提高流表性能就是提升网络性能的重要措施之一。详细介绍 SDN 架构下流表理论基础及定义, 详细介绍目前当前网络设备基于 TCAM 硬件和基于 Hash 软件两种流表的工作原理, 结合 DPDK 技术特性和网络数据处理的优势, 以及 APT 攻击检测总体架构提出基于 DPDK 的二级核间负载均衡软硬协同架构, 即 RSS 硬件多核负载均衡和 Cockoo Hash 软件流表组成的混合架构。通过对 RSS 硬件多核负载均衡进行压力测试, 以及 Hash 算法仿真实验的方式, 测试 Fowler-Noll-Vo Hash 算法、Jenkins Hash 算法、Paul Hsieh Hash 算法、OAT Hash 算法、Bernstein Hash 算法、SAX Hash 算法, 基于 Cockoo Hash 的 CRC Hash 算法和 JHash 算法在同样的硬件环境下, 在特定 APT 攻击流量特征对于流表的增、查、删仿真压力测试, 对比得出最适用于 APT 攻击场景下的 Hash 算法, 按照 Cockoo Hash 的理论基础, 以及 Paul Hsieh Hash 算法查找性能适中、稳定性强的特性, 将 Paul Hsieh Hash 算法融入 Cockoo Hash 算法结构中, 优化后算法命名为基于 Cockoo Hash 的 Paul Hsieh Hash 算法。

实验结果表明, 基于 DPDK 架构的 RSS 硬件多核负载均衡后流量在多核间实现对称分流, 多核并行计算下网络性能远远高于非 RSS 硬件状态下的网络性能。同样基于 Cockoo Hash 的 Paul Hsieh Hash 算法的软件流表性能在 APT 攻击场景下对于流表的增、查、删压力测试中, 有更优异的性能表现, 证明基于 DPDK 的二级核间负载均衡软硬协同作为 APT 攻击流量检测优势。

第4章 基于DPDK的APT攻击特性研究

4.1 APT攻击理论基础及定义

APT攻击者往往以组织的形态出现，从锁定目标到最终得手，会经过多个精心策划的阶段。其中，洛克希德-马丁公司提出网络攻击杀伤链 Cyber-Kill-Chain 模型涵盖了七个关键环节，信息收集、武器构建、载荷投送到目标网络、漏洞利用、安装植入恶意软件、命令控制恶意软件建立通信信道、实现攻击目标瘫痪或渗透进入系统。

MITRE 是美国政府资助的一家研究机构，该机构 1958 年从 MIT 的林肯实验室分离出来参与了许多商业和最高机密项目，其中包括多种空中管制系统、雷达系统和防空系统等。MITRE 在美国国家标准技术研究所的资助下从事了大量的网络安全实践。MITRE 在 2013 年推出了 ATT&CK 模型，ATT&CK 模型通过模拟网络攻击者身份来描述 APT 攻击在各个阶段的攻击战略、战术的模型，ATT&CK 模型如图 4-1 所示。

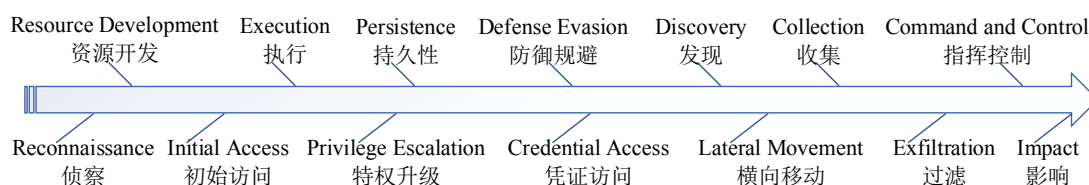


图 4-1 ATT&CK 模型

ATT&CK 模型围绕的 TTP 概念最初由美国国防部提出，可认为是一种对军事活动进行规范性描述的方法论框架。自概念引入网络安全领域起，TTP 就经常被用于描述入侵攻击活动中攻击者的行为和活动方式。TTP 主要以两个方面展开，即战术（Tactics）与技术（Techniques），所以网络安全的核心也是这两方面，成功的网络安全中战术起到核心作用。战术是攻击者采取的攻击策略，包括侦察（Reconnaissance）、横向移动（Lateral Movement）、命令与控制（C&C）等。技术是攻击者为执行既定的攻击策略，所采用的技术手段，如对应 C&C 战术中加密通道（Encrypted Channel）、数据混淆（Data Obfuscation）等技术。

ATT&CK 已确定了三个技术领域 Enterprise、Mobile 和 ICS，本文研究所涉及 ATT&CK 是指 ATT&CK for Enterprise，模型广泛应用于企业网络安全，主要用于 APT 入侵检测与防御、实时监控网络流量和识别等，实现阻止攻击行为，快速应对网络攻击，减少潜在损害。互联网的普遍使用导致网络攻击显着升级，给机构和个人都带来了严重的有害后果。由于网络攻击的频繁发生和复杂性的增加，当代网络企业被迫采用严格的安全协议，以确保数据传输的完整性。

4.2 APT 攻击检测的目标与挑战

4.2.1 APT 攻击检测的目标与挑战

结合本文研究的基于 DPDK 的 APT 攻击检测网关，围绕网络入侵检测，通过分析总结 ATT&CK 模型 APT 攻击特征及各个阶段特点，将 APT 攻击检测网关能够实现的战术形成防御策略，研究现状，进行梳理整合和归纳总结。

在 APT 攻击的侦察阶段，攻击者会从外部网络对目标网络进行主动扫描，意图获取主机的 IP 地址，以收集可在定位期间使用的信息，例如物理位置、网络服务提供商以及托管其面向公众的基础设施，扫描开放端口号、操作系统类型等一般配置信息以及最重要的脆弱性信息，攻击者还可能针对有关基于受害者网络的入侵检测系统（Network Intrusion Detection System, NIDS）或与防御性网络安全操作相关的其他设备的信息。对于大多数的 APT 攻击而言，侦察阶段所花费的时间占据整个 APT 攻击大部分时间。因此，对网络防御者而言，如何采取措施防止攻击者侦查到目标网络的配置和脆弱性信息，是保护整个目标网络安全的关键。

在 APT 攻击经过前期的侦查和准备，攻击者在凭据访问中滥用可以确定网络流量的常见网络协议，采用 LLMNR/NBT-NS 中毒、SMB（Server Message Block）中继、ARP 缓存中毒、DHCP 欺骗等技术，获取重要数据及高价值目标节点的访问权限的攻击行为，嗅探网络流量以捕获有关环境的信息，包括通过网络传递的身份验证材料。网络嗅探是指通过网络接口，捕获有线网络或无线网络传输的网络数据报文。通常网络嗅探需要将网络接口配置为混杂模式，混杂模式可以捕获大量目的非本地的网络数据，或者使用专用网络捕获硬件，或者类似于 DPDK 的技术捕获网络数据。传统的防御手段往往难以检测到攻击者所处的位置且具有一定的滞后性，难以有效开展防御。因此，如何确定攻击者的位置并打断其攻击进程，是防御 APT 攻击中期阶段攻击行为的主要手段。

在 APT 攻击命令与控制阶段，攻击者可以使用 OSI 协议对远程系统的命令的结果嵌入到客户端和服务端之间的正常流量混合来避免检测过滤。攻击者可能会将垃圾数据添加到用于命令和控制的协议中，以使检测更加困难。通过将随机或无意义的垃圾数据添加到用于命令和控制的协议中，攻击者可以阻止用于解码、破译或以其他方式分析流量的琐碎方法。包括使用垃圾字符预置数据或在有效字符之间写入垃圾字符。攻击者可能会在单独的协议中通过隧道与受害系统进行网络通信，以避免网络检测对其他无法访问系统的访问。隧道涉及将一个协议显式封装在另一个协议中。此行为可能会通过与现有流量混合来隐藏恶意流量。隧道还可以路由网络数据包，否则这些数据包将无法到达其预期目

标。因此在该阶段，防御者需要重点针对目标节点开展防御。

最后在影响阶段实施 APT 攻击时，包含多种攻击战术，例如 DOS/DDoS 攻击致瘫目标网络，主要包括以下几种攻击方式^[38]。

(1) SYN Flood 向目标服务器发送大量 SYN 请求包但未完成 TCP 三向握手，从而耗尽服务器的连接资源。

(2) ICMP Flood 向目标服务器发送大量的 ICMP 请求数据包，导致服务器忙于响应这些请求并耗尽其资源。

(3) UDP Flood 向目标服务器上的随机端口发送大量 UDP 数据包，迫使服务器不断检查并响应这些不存在的请求，从而消耗服务器的计算资源。

传统的 DDoS 攻击通常可以通过流量限速、黑名单过滤、防火墙规则等方法来有效缓解。然而随着新型 DDoS 攻击的出现，例如使用欺骗性 IP 技术的攻击，针对依赖于广泛数据分析的新型 DDoS 攻击防御方法。这些方法需要分析大量网络数据来识别异常流量模式并区分合法流量和攻击流量，对数据的处理能力提出了很高的要求，从而导致巨大的资源消耗和性能瓶颈。

4.2.2 基于 DPDK 的 Snort 入侵检测系统

研究高性能网络架构下对 APT 攻击流量的实时检查和分析，针对 APT 攻击检测解决方案，具体措施有三个方向：

(1) 网络隔离、访问控制、加密等策略和措施^[39]。网络隔离是指与其他网络隔离，防止 APT 攻击蔓延。这种方式可以缩小攻击范围，但也降低了系统的灵活性和可用性，并且需要更多的管理和维护成本。访问控制是指对用户和设备进行身份验证并控制其访问以确保系统安全。这种方式可以提供一定的安全保障，但需要维护和管理大量的认证信息，增加了管理成本。加密技术是指利用加密技术来保护数据的传输和存储。这种方式可以提供一定程度的安全保障，但对系统性能造成一定的影响。加大通信系统的时延和消耗。此外，攻击者可能使用更先进的技术破解密码的风险。

(2) 检测技术包括基于网络流量监控、异常检测、攻击图、行为分析和人工智能的方法。基于网络流量监控的方法通过监控网络流量来检测和分析网络中的异常数据流和行为，以识别可能的 APT 攻击^[40]。然而，这种方法取决于流量监测的准确性和效率。如果攻击者采用分阶段攻击、伪造流量等隐蔽攻击方式，这种方法可能无法有效检测攻击。基于异常检测的方法对系统设备和用户的行为进行建模，并检测异常行为以识别可能的 APT 攻击。该方法需要足够的数据训练和建模，还需要处理大量的误报和漏报。它还需要克服攻击者采用隐蔽攻击的挑战。基于攻击图的方法通过分析攻击者对系统的攻击路径和目标来

识别可能的 APT 攻击。该方法需要对系统进行全面的安全建模和攻击图分析，但系统中的组件和连接往往非常复杂。因此，该方法的建模和分析工作需要较高的人力和时间成本。基于行为分析的方法分析设备、用户和网络的行为并检测任何异常行为。基于行为分析的方法可能具有较高的误报率，因为正常行为可能被误认为异常行为，从而增加管理和维护成本。人工智能使用机器学习和深度学习算法来监控系统的行为并检测异常活动。机器学习和深度学习算法可能会产生误报或漏报，因此需要不断改进和训练算法。总之，这些检测方法需要在网络环境中提供足够的标记数据，这增加了成本。此外，攻击者还可以利用对抗技术来欺骗检测方法，进一步增加防御 APT 的难度。

(3) 使用硬件安全、防火墙和漏洞管理来确保设备和数据的安全^[41]。硬件安全是指利用硬件设备和安全芯片来保护网络的安全。这种方法安全性较高，但成本较高，且存在供应链攻击和物理攻击的风险。

本文解决方案结合上述三个方向优势，第 2 章基于 DPDK 的攻击检测网关总体架构集成 TEE 硬件设备安全存储关键信息，实现网络安全域与非安全域隔离，第 3 章二级核间负载均衡架构优化流表访问控制、加密等策略查找效率，构建 L2TP/IPSec 加密隧道通道保护数据的传输和存储，使用 SDN 控制平面分离架构降低维护和管理成本，并结合本章入侵检测内容。

本章所研究 IDS 通过监视网络以检测和识别未经授权的活动或违反既定策略的硬件或软件实体。IDS 主要研究包括基于签名的检测和基于异常的检测。基于签名的检测依赖已知的签名数据库，IDS 通过基于签名的模式匹配来识别恶意的 C&C 网络流量。如果检测到入侵，IDS 会捕获有关该事件的相关数据、发布警报并根据需要执行适当的补救或预防措施。因此，IDS 已成为任何安全框架不可或缺的组成部分。

Snort 是出色的基于签名的网络入侵检测系统^[42]。Snort 目前可以运用大量网络入侵检测的规则，这些规则在识别恶意的 C&C 网络流量和识别可疑网络活动是许多行之有效的案例，Snort 结合这些规则可以用于检测 APT 攻击流量中存在的潜在威胁。采用静态的规则调整算法是目前减少重复性 OTN (Option Tree Node) 匹配过程的主流方案之一。该算法主要是采用统计分析 Snort 检测日志中对各个规则的匹配频率，然后根据正态分布的方式来调整 OTN 在规则链表中的顺序，使匹配频率高的 OTN 的位置提前优化了系统性能。基于 DPDK 的攻击检测总体架构及二级核间负载均衡软硬协同架构下构建基于 DPDK 的 Snort 入侵检测系统，提高 APT 防御能力。

4.3 APT 攻击检测仿真实验

4.3.1 APT 攻击检测仿真测试实验环境搭建

硬件运行环境见表 4-1，软件运行环境见表 4-2。实验环境采用与第 2 章节相同硬件运行环境，相同操作系统，相同测试工具，集成第 2 章基于 DPDK 的 APT 攻击检测的总体架构，第 3 章基于 DPDK 的二级核间负载均衡软硬协同架构，即 RSS 硬件多核负载均衡和 Cockoo Hash 软件流表组成的混合架构，第 4 章研究成果，构建模拟 APT 攻击检测集成验证环境，测出实验数据。

表 4-1 硬件运行环境与测试单元的关系

CPU	DDR4	操作系统	测试单元
国产申威 3231CPU	16GB	统信 UOS	模拟攻击测试
Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz	16GB	Ubuntu 22.04.4 LTS	模拟攻击测试

表 4-2 软件运行环境与功能的关系

软件运行环境/依赖	功能	软件/内核版本
GCC	编译工具	8.3.0
Python	脚本工具	3.10.12
DPDK	数据平面	19.11
Snort	APT 攻击检测	3.0.0
tcpreplay	模拟攻击工具	4.3.4
DDOS_Attack	模拟 DDOS 攻击	1.0.0
Wireshark	网络协议分析工具	4.2.0
Ubuntu 22.04.4 LTS	操作系统	Linux 5.15.146.1 #1 SMP x86_64
	(模拟攻击者)	GNU/Linux
统信 UOS V18	操作系统	Linux 4.19.0-sw64-server #225 SMP
	(被测)	sw64 GNU/Linux

由于 Snort 原生代码程序嗅探报文是通过传统混杂模式网卡进行接收，与 DPDK 架构收发报文方式不一致，所以首先将 Snort 入侵检测软件移植适配 DPDK 架构，融入基于 DPDK 的攻击检测网关总体架构攻击检测模块中，网络报文在负载均衡模块后进行入侵检测本文研究 APT 攻击检测网关需要在内部网络出口处部署，实现网关端点处阻断攻击，保护内部网络。

APT 攻击实际网络环境部署拓扑图如图 4-2 所示，数据平面与管理平面分离，APT 攻击检测网关在外网未知安全域建立 VPN 隧道加密内部外出数据报

文，内网安全域数据流动为确认安全后的明文数据。攻击者在远端自治系统（Autonomous System, AS）通过路由器延红色网络路径发起网络攻击，意图对内网用户进行实施 APT 攻击。

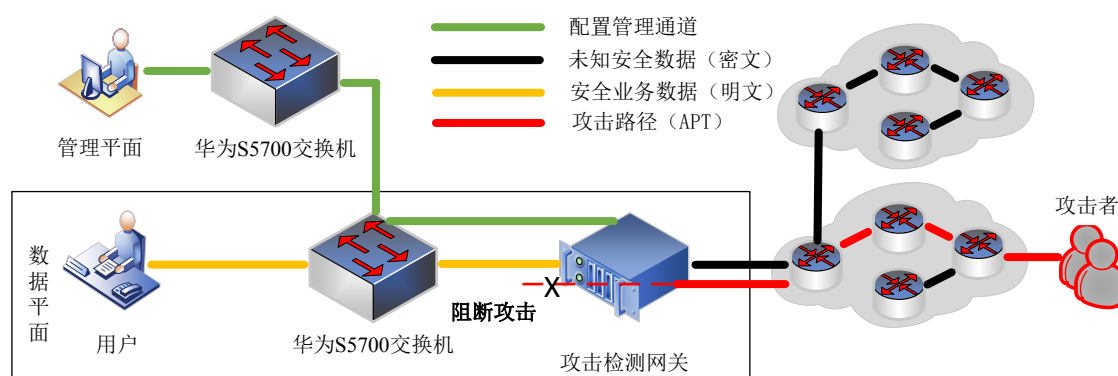


图 4-2 APT 攻击实际网络环境部署拓扑图

4.3.2 APT 攻击检测仿真测试实验

KDD99 是入侵检测系统经典的数据集，该数据集是自 1999 年以来评估入侵检测系统最常见和标准的数据集，是基于 DARPA98 项目记录的数据。该数据集包含由 tcpdump 软件从网络流量中收集的大约 4GB 的原始二进制数据记录。采用实验数据集信息见表 4-3 所示。

表 4-3 实验数据集信息

数据集	记录条数	目标数据类型
CAIDA	约 700 万条	网络流量数据集
KDD99	约 50 万条	入侵流量数据集
CIC-IDS2017-PCAP	约 120 万条	入侵流量数据集

本实验使用 APT 数据集作为实验测试的数据源，Snort 配置最新网络安全规则策略，使用最新基线模型，模拟攻击者在 Linux 环境下安装 tcpreplay 工具，用于回放实验数据集。模拟攻击者在 APT 攻击检测网关的外网处，直接发起 APT 攻击，随机采用实验数据集 1% 的数据报文作为 APT 攻击流量进行重放。

APT 攻击实验模拟部署拓扑图如图 4-3 所示，根据 APT 攻击实际网络环境部署拓扑主干环境搭建模拟环境，使用网络测试仪代替用户模拟内网明文业务流量，在华为 S5700 交换机和攻击检测网关之间搭建 VPN 隧道 L2TP/IPSec 服务，加密算法采用高级随机数生成素数（AES）和对称加密算法（RSA）相结合，模拟外网密文业务流量。

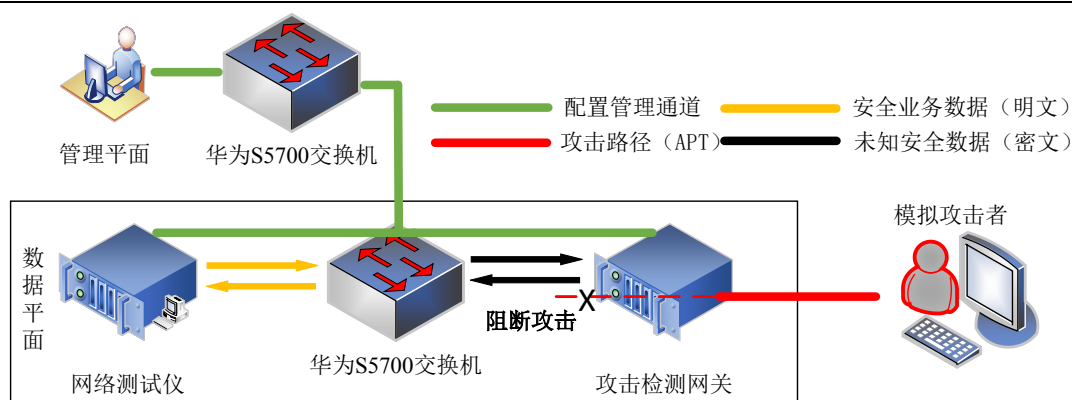


图 4-3 APT 攻击实验模拟环境部署拓扑图

本实验模拟攻击者除了使用上述实验数据源外，模拟攻击者利用攻击路径剩余带宽，模拟 DDOS 的 UDP Flood 攻击消耗目标基于 DPDK 的 APT 攻击检测网关处理性能，增加入侵流量检测难度。在 APT 攻击检测网关外网处使用 Wireshark 捕获到的 UDP Flood 如图 4-4 所示。

No.	Time	Source	Destination	Protocol	Length	Info
43655	30.652987	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23060 Len=1490
43656	30.653054	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e53) [Reassembled in #43657]
43657	30.653057	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23061 Len=1490
43658	30.653171	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e54) [Reassembled in #43659]
43659	30.653174	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23062 Len=1490
43660	30.653231	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e55) [Reassembled in #43661]
43661	30.653234	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23063 Len=1490
43662	30.653259	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e56) [Reassembled in #43663]
43663	30.653261	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23064 Len=1490
43664	30.653307	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e57) [Reassembled in #43665]
43665	30.653310	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23065 Len=1490
43666	30.653359	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e58) [Reassembled in #43667]
43667	30.653362	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23066 Len=1490
43668	30.653411	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e59) [Reassembled in #43669]
43669	30.653413	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23067 Len=1490
43670	30.653465	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e5a) [Reassembled in #43671]
43671	30.653467	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23068 Len=1490
43672	30.653519	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e5b) [Reassembled in #43673]
43673	30.653521	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23069 Len=1490
43674	30.653573	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e5c) [Reassembled in #43675]
43675	30.653576	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23070 Len=1490
43676	30.653599	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e5d) [Reassembled in #43677]
43677	30.653600	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23071 Len=1490
43678	30.653646	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e5e) [Reassembled in #43679]
43679	30.653648	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23072 Len=1490
43680	30.653698	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e5f) [Reassembled in #43681]
43681	30.653701	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23073 Len=1490
43682	30.653717	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e60) [Reassembled in #43683]
43683	30.653719	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23074 Len=1490
43684	30.653733	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e61) [Reassembled in #43685]
43685	30.653735	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23075 Len=1490
43686	30.653781	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e62) [Reassembled in #43687]
43687	30.653784	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23076 Len=1490
43688	30.653801	172.23.17.41	192.168.22.1	IPv4	1514	Fragmented IP protocol (proto=UDP 17, off=0, ID=9e63) [Reassembled in #43689]
43689	30.653804	172.23.17.41	192.168.22.1	DNPv20	52	33184 → 23077 Len=1490

图 4-4 模拟 DDOS 攻击流量 Wireshark 示意图

网络测试仪操作如第 2.4 章节架构网络性能测试实验相同，操作网络测试仪发送流量模拟正常业务流量。配置网络流量数量为一条，方向改为双向，执行 RFC2544 吞吐量测试。受到攻击时基于 DPDK 的 APT 攻击检测架构双向转发性能结果如表 4-4 所示。

测试数据如表 4-4，图 4-5 所示，在受到网络攻击的环境下，大量攻击流量抢占正常网络带宽，所以在没有网络安全设备保护的情况下，网络测试仪模拟

的业务流量吞吐量由于背景流量中的 DDOS 攻击, 相较正常网络环境吞吐量大幅度降低, 测试结果约在 100Mbps-300Mbps 之间, 90%以上的性能损耗, 全方面致瘫网络。

表 4-4 受到攻击时基于 DPDK 的 APT 攻击检测架构双向转发性能测试汇总表

帧长 L (Bytes)	目标负载 O_t (%)	实际负载 O_r (%)	吞吐量 TP (%)	吞吐量 TP (fps)	吞吐量 TP (Mbps)
64	4.938	4.938	4.938	146940	987.599
128	8.875	8.875	8.875	1499154	1773.999
256	17.032	17.032	17.032	1542752	3406.399
512	33.204	33.204	33.204	1560338	6640.8
1024	59.922	59.922	59.922	1434912	11984.4
1280	81.719	81.719	81.719	1571518	16343.8

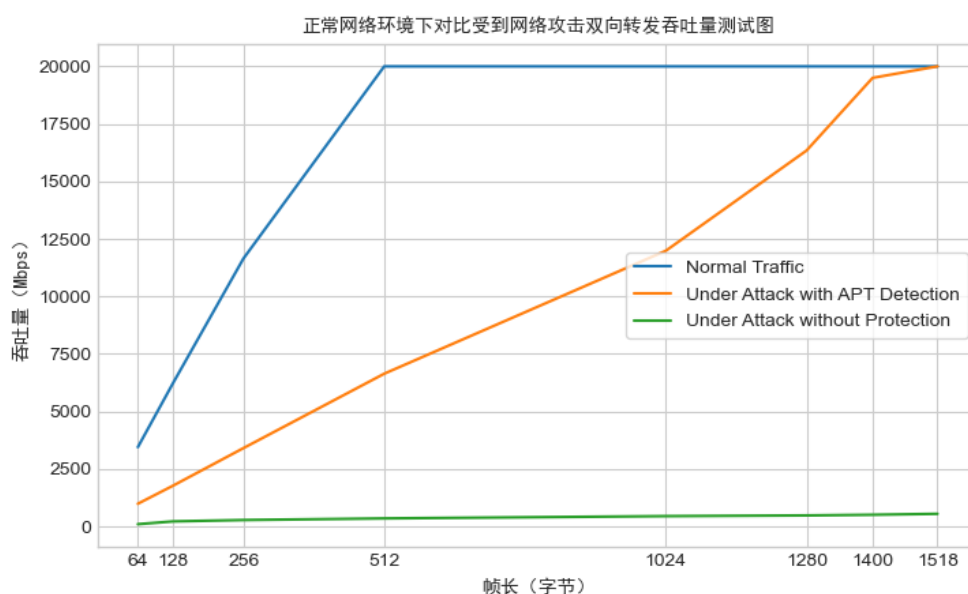


图 4-5 正常网络环境下对比受到网络攻击双向转发吞吐量测试图

但是, 在部署 APT 攻击检测网关的网络环境中, 虽然短报文业务流量吞吐量仍有 40%-50%性能损耗, 但是依然具备比较好的网络处理能力, 平均时延 100-200us, 在长报文业务流量吞吐量可以看到几乎没有损耗, 实验数据表明, APT 攻击检测网关具备非常优异攻击检测能力, 并且在实时防御的同时保证良好的网络性能。

4.4 本章小结

本章以课题研究的 APT 攻击特征为中心,围绕 ATT&CK 模型详细介绍 APT 攻击理论及定义,针对现阶段 APT 攻击检测的目标与挑战,形成 APT 攻击检测解决方案,以 IDS 为 APT 攻击检测模块主体,将 Snort 融入 DPDK 技术架构。集成第 2 章到第 4 章研究成果,部署基于 DPDK 的 APT 攻击检测网关,构建模拟 APT 攻击检测实验环境,使用入侵检测数据集作为实验的数据源,同时在攻击路径引入 DDOS 攻击消耗 APT 攻击检测网关性能,测出实验数据。

实验结果表明,相比较无网络安全设备保护的情况下完全致瘫网络,基于 DPDK 的 APT 攻击检测网关具备优秀的攻击检测能力,在吞吐量方面则有非常明显的提升。从整体来看,大幅度降低 APT 攻击对于内部网络正常业务的影响。

第 5 章 结论与展望

5.1 全文总结

随着全球冲突、攻防对抗的不断升级，网络威胁背后是由国家背景支持，APT 威胁活动往往与国家利益存在关联，历年来政府机构都是 APT 组织首要攻击目标，尤其是在国家关键基础设施（Critical Information Infrastructure, CII）等重要领域，可能导致严重的后果。本文研究旨在探索一种有效的网络防御方案，以应对日益复杂和隐蔽的 APT 威胁。

本文主要工作包含以下几个方面：

（1）**基于 DPDK 的 APT 检测架构设计** 本文提出了一种结合 SDN 技术的 APT 检测网关架构。分析了传统网络架构在面对网络攻击时的局限性，分析了 SDN 技术如何为网络安全提供新的解决方案。研究关注 SDN 数据平面，架构设计重点为数据平面网络数据处理能力、数据平面架构级安全隔离性和数据平面网络攻击检测扩展性。提出基于英特尔 DPDK 框架的 APT 检测架构，管理平面和数据平面分离，功能接口隔离未知安全域与安全域。结合 DPDK 在多核处理器中网络数据处理优势，可以实现对攻击流量的实时监控和处理。实验数据表明，该架构能够显著提升网络设备性能，具备良好的网络攻击防御基础，同时兼顾网络设备的扩展性和可靠性。

（2）**流表性能优化方案** 深入分析了网络攻击流量对流表项性能的影响，提出了一种基于硬件和软件双重流表的混合架构。本文提出了在总体架构下基于 DPDK 的二级核间负载均衡软硬协同架构，即 RSS 硬件多核负载均衡和 Cuckoo Hash 软件流表组成的混合架构。该方案通过在硬件层面实现数据包分流提高并行流量处理能力，在软件层面加速流表查找效率。实验使用网络攻击报文数据集，结合 APT 攻击流量特征测试八种 Hash 算法及基于 Cuckoo Hash 理论优化后算法，并通过对 RSS 硬件多核负载均衡进行压力测试，实验数据表明，该方案有效提高网络实时性和攻击检测扩展性的需求。

（3）**APT 攻击检测** 基于 DPDK 架构的 APT 检测方案在网络攻击流量下的表现优于传统方法。具体而言，该架构集成硬件安全、物理隔离、网络性能提升、流表效率提升，能够以较低的延迟率实现高效的攻击流量识别，同时在大规模网络环境下展现出良好的可扩展性。通过具体的仿真实验验证了提出的架构在实际应用中的可行性，并评估了其在网络攻击环境下的性能表现。

作为一种新型网络防御架构，本文提出的基于 SDN 的 APT 检测方案为相关领域提供了新的思路和解决方案，通过将攻击流量在多核处理器并行计算，

可以有效减少对网络资源的影响，从而提升系统的整体性能和可靠性。以及针对 APT 攻击流量特征提出的基于 DPDK 的二级核间负载均衡软硬协同架构，即 RSS 硬件多核负载均衡和 Cuckoo Hash 软件流表组成的混合架构，不仅能够为网络安全领域提供新的技术手段，还能够提升关键信息基础设施的安全防护水平，通过降低设备成本从而降低潜在安全风险的应用成本。其次，本文的实验结果能够为网络设备制造商、网络运营商以及科研机构等实际应用场景提供参考，帮助他们在面对 APT 威胁时做出更科学的决策。

本文研究内容是网络安全技术综合应用，对增强网络空间防护能力，保障基础设施安全，提高攻击响应速度，应对新型攻击威胁等方面有很重要的实用意义。这将有助于构建更加安全、高效的网络环境，为保护国家安全、企业安全和个人安全提供坚实的技术支撑。

5.2 研究展望

在本文所研究的工作基础之上，对于后续的研究工作可以从以下多个角度着手去探索：

（1）**虚拟化部署** 针对本文基于 DPDK 的 APT 攻击检测架构，VNF 单个占用的资源粒度过大，系统空闲时设备资源浪费严重。如何进一步将该架构虚拟化部署，保证应用业务正常、网络安全的同时，避免设备资源过度占用，从而实现整体网络功能虚拟化。实验主要依赖于模拟环境，实际应用中可能会受到硬件设备性能的限制。

（2）**扩大实验规模** 仿真实验虽然能够验证架构的基本性能，但无法完全模拟实际网络环境中的复杂因素。进一步探索在大规模网络实际环境中的表现，扩大网络智能化层面，提高 APT 攻击检测对于复杂网络环境适应能力。

（3）**算法优化空间** 针对本文基于 Hash 算法数据结构软件流表项存储方案，如何进行进一步地研究专用算法，结合多种数据结构（如布谷鸟哈希表、红黑树等）构建更高效的网络安全架构，在保证收敛性的同时，加快算法的收敛速度，以提升系统的实时性和可靠性。

（4）**提高 APT 攻击检测准确性** 融合人工智能（Artificial Intelligence, AI）的智能化和自适应性，使其能更好地适应 DPDK 环境，通过大量的训练 AI 模型以优化其决策机制，大数据分析和模式识别技术处理和分析大量数据，降低误报率，以期在保证高检测效率的同时，有效地提升网络系统的安全防御能力。

参考文献

- [1] 蒲石, 陈周国, 祝世雄. 震网病毒分析与防范[J]. 信息安全, 2012(2): 40-43.
- [2] National Institute of Standards and Technology. Managing Information Security Risks: NIST SP 800-53[R]. Gaithersburg, MD: National Institute of Standards and Technology, 2013.
- [3] Joint Task Force Transformation Initiative. Managing Information Security Risk: Organization, Mission, and Information System View: NIST SP 800-39[R]. Gaithersburg, MD: National Institute of Standards and Technology, 2011.
- [4] Case D U. Analysis of the cyber attack on the Ukrainian power grid[J]. Electricity Information Sharing and Analysis Center (E-ISAC), 2016(1-29): 3.
- [5] Qi An Xin Technology Group Inc. Global advanced persistent threats(APT) mid-2023 report[R]. Beijing: Qi An Xin Technology Group Inc., 2023.
- [6] 《中国网信》杂志发表《习近平总书记指引我国网络安全工作纪实》[J]. 工业信息安全, 2022(8): 83-90.
- [7] Khan M T, Deblasio J, Voelker G M, et al. An empirical analysis of the commercial VPN ecosystem[C]//Proceedings of the 2018 Internet Measurement Conference. Boston, MA: ACM, 2018: 443-456.
- [8] Tolley W J, Kujath B, Khan M T, et al. Blind in/on-path attacks and applications to VPNs[C]//30th USENIX Security Symposium (USENIX Security 21). Online: USENIX Association, 2021: 3129-3146.
- [9] Attig M, Brebner G. 400 Gb/s programmable packet parsing on a single FPGA[C]//2011 ACM/IEEE Seventh Symposium on Architectures for Networking and Communications Systems. Brooklyn, NY: ACM/IEEE, 2011: 12-23.
- [10] Sha M, Guo Z, Wang K, et al. A High-Performance and Accurate FPGA-Based Flow Monitor for 100 Gbps Networks[J]. Electronics, 2022, 11(13): 1976.
- [11] 吴克河, 郑碧煌, 张玉俊, 等. 基于 PF_RING 的电力安全网关设计与实现[J]. 电力信息与通信技术, 2022, 20(09): 9-15.
- [12] 刘磊, 孔志印, 赵荣彩, 等. 基于多核的 IPSec 并行处理技术研究 with 实现[J]. 计算机工程, 2009, 35(03): 139-141.
- [13] Intel. Data plane development kit (DPDK)[EB/OL]. (2024-01-01)[2024-05-06]. <https://www.dpdk.org/>.
- [14] Fu C, Wang B, Wang W, et al. A Generic High-Performance Architecture for VPN Gateways[J]. Electronics, 2024, 13(11): 2031.
- [15] Lei C, Zhang H Q, Tan J L, et al. Moving Target Defense Techniques: A

- Survey[J]. Security and Communication Networks, 2018, 2018: 1-25.
- [16] Gao Z, Chen F, Wang Y, et al. MVPN: A Defense Architecture against VPN Traffic Hijacking Based on MTD[J]. Electronics, 2023, 12(3): 711.
- [17] Abu Bakar R, Kijirikul B. Enhancing Network Visibility and Security with Advanced Port Scanning Techniques[J]. Sensors, 2023, 23(17): 7541.
- [18] 李键. SDN 环境下的 DDoS 攻击检测溯源技术研究[D]. 电子科技大学. 2024.
- [19] 赵扬. SDN 数据平面异常行为检测方法研究[D]. 战略支援部队信息工程大学. 2022.
- [20] 林家皓. 基于 DPDK 的流量重组及分布式存储技术研究[实现][D]. 电子科技大学. 2021.
- [21] 杨文韬. 基于大象流卸载的高速软硬协同流表设计与实现[D]. 东南大学. 2022.
- [22] 李猛, 罗文啟, 戴海鹏, 等. 代价敏感的指纹可变哈希布谷鸟过滤器[J/OL]. 软件学报: 1-18[2024-07-05]. <https://doi.org/10.13328/j.cnki.jos.007221>.
- [23] 李元诚, 罗昊, 王庆乐, 等. 一种基于 ATT&CK 的新型电力系统 APT 攻击建模[J]. 信息安全, 2023, 23(02): 26-34.
- [24] Gan C, Lin J, Huang D W, et al. Advanced Persistent Threats and Their Defense Methods in Industrial Internet of Things: A Survey[J]. Mathematics, 2023, 11(14): 3115.
- [25] 王郅伟, 何晞杰, 易鑫, 等. 基于 APT 活动全生命周期的攻击与检测综述[J]. 通信学报, 2024, 45(09): 206-228.
- [26] 孔冠骅. 面向 APT 攻击的网络欺骗防御策略研究[D]. 战略支援部队信息工程大学. 2024.
- [27] Zhao G, Xu K, Xu L, et al. Detecting APT Malware Infections Based on Malicious DNS and Traffic Analysis[J]. IEEE Access, 2015, 3: 1132-1142.
- [28] Barghash A, Hammad L, Gharaibeh A. Traditional vs. Modern Data Paths: A Comprehensive Survey[J]. Computers, 2022, 11(9): 132.
- [29] Bhargavi G. Experimental based performance testing of different TCP protocol variants in comparison of RCP+ over hybrid network scenario[J]. International Journal of Innovative and Advanced Computer Science, 2014, 3(1): 2347-8616.
- [30] Li Z. HPSRouter: A high performance software router based on DPDK[C] //Proceedings of the 2018 20th International Conference on Computer Communications. Chuncheon-si, Korea: IEEE, 2018.
- [31] Rizzo L. Netmap: A novel framework for fast packet I/O[C]//21st USENIX Security Symposium (USENIX Security 12). Bellevue, WA: USENIX Association, 2012: 101-112.
- [32] Lai L, Ara G, Cucinotta T, et al. Ultra-low latency NFV services using

- DPDK[C]//2021 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN). Online: IEEE, 2021.
- [33] 金燕, 陈佳彬, 陈尔言, 等. 基于 RFC2544 的千兆以太网性能测试的实现[J]. 浙江工业大学学报, 2016, 44(06): 628-632.
- [34] Network Working Group. RFC 2544: Benchmarking methodology for network interconnect devices[S/OL]. Fremont, CA: Internet Engineering Task Force (IETF), 1999[2024-05-06]. <https://www.rfc-editor.org/rfc/rfc2544>.
- [35] Isyaku B, Mohd Zahid M S, Bte Kamat M, et al. Software Defined Networking Flow Table Management of OpenFlow Switches Performance and Security Challenges: A Survey[J]. Future Internet, 2020, 12(9): 147.
- [36] Alahmadi A, Chung T S. An Effective Selection of Memory Technologies for TCAM to Improve the Search Operations: Demonstration of Memory Efficiency in SDN Recovery[J]. Electronics, 2024, 13(4): 707.
- [37] Microsoft. Introduction to receive side scaling[EB/OL]. (2023-02-20)[2023-03-12]. <https://learn.microsoft.com/en-us/windows-hardware/drivers/network/introduction-to-receive-side-scaling>.
- [38] Dong S, Abbas K, Jain R. A survey on distributed denial of service (DDoS) attacks in SDN and cloud computing environments[J]. IEEE Access, 2019, 7: 80813-80828.
- [39] Qi L, Yang Y, Zhou X, et al. Fast anomaly identification based on multiaspect data streams for intelligent intrusion detection toward secure industry 4.0[J]. IEEE Transactions on Industrial Informatics, 2021, 18: 6503-6511.
- [40] Vijayakumaran C, Muthusenthil B, Manickavasagam B. A reliable next generation cyber security architecture for Industrial Internet of Things environment[J]. International Journal of Electrical and Computer Engineering, 2020, 10: 387.
- [41] Coppolino L, D'Antonio S, Mazzeo G, et al. A comprehensive survey of hardware-assisted security: From the edge to the cloud[J]. Internet of Things, 2019, 6: 100055.
- [42] Sourcefire. Snort network intrusion detection system[EB/OL]. (2024)[2024-05-06]. <https://www.snort.org/>.